Ollscoil Teicneolaíochta an Atlantaigh

Atlantic Technological University

Bachelor of Engineering (Hons) in Software and Electronic Engineering

FINAL YEAR PROJECT REPORT

Smart System Performance Dashboard

Real-time Hardware Monitoring with ML-Powered Fault Detection for Windows

Andrew Fox

Student ID: G00417373

Supervisor: Brian O'Shea

Project Engineering

Academic Year 2025 / 2026

Declaration

This project is presented in partial fulfilment of the requirements for the degree of Bachelor of Engineering (Honours) in Software and Electronic Engineering.

This project is my own work, except where otherwise accredited. Where the work of others has been used or incorporated, it is duly acknowledged in the text.

Any use of AI tools (such as ChatGPT or GitHub Copilot) is acknowledged in Appendix A and was limited to minor tasks such as debugging assistance and asking questions.

Acknowledgements

I would like to thank my supervisor Brian O'Shea for his guidance and feedback throughout both semesters. Thanks also to my classmates for the active discussions during stand-ups and project reviews, and to my parents for their continuous support throughout the years.

Table of Contents

Declaration 2
Acknowledgements 3
Table of Contents 4
Summary 6
Poster 7
1. Introduction 8
1.1 Report Structure 8
2. Background and Research 10
2.1 Existing System Monitoring Tools 10
2.2 Technology Selection and Justification 10
2.2.1 Python 10
2.2.2 PyQt6 and pyqtgraph 11
2.2.3 psutil, pynvml, and pywin32 11
2.2.4 SQLite with WAL Mode 11
2.2.5 scikit-learn Random Forest 12
2.3 Requirements 12
3. System Architecture 14
3.1 The Four Services 14
3.1.1 Collector Service (QThread, 1-second loop) 14
3.1.2 Storage Service (SQLite, WAL Mode) 15
3.1.3 Analytics Service (ML Inference, 5-second cycle) 15
3.1.4 Dashboard Service (PyQt6 GUI Frontend) 15
3.2 Key Design Decisions 16
4. Project Plan and Development Methodology 17
4.1 Agile Scrum Methodology 17
4.2 Version Control 17
5. Live System Monitoring 20
5.1 Data Collection Backend 20
5.2 Performance Optimisation 21
5.3 GUI Implementation 22
5.4 Real-Time Graphing 23
6. Database and Storage Service 24
6.1 Design Rationale 24
6.2 Schema Design 24
6.3 Storage Service Implementation 26
6.4 Training Data Collection 26
7. Machine Learning and Analytics 27
7.1 Model Selection 27

7.2 The 12 Detected Fault Categories 27
7.3 Feature Engineering 27
7.4 Label Generation 28
7.5 Training Data 28
7.6 Model Training and Results 29
7.7 Inference and Dashboard Integration 32
7.8 Health Score Formula 33
8. Testing 35
8.1 Test Strategy 35
8.2 Collector Tests (test_collectors.py, 310 lines) 35
8.3 Storage Tests (test_storage.py, 168 lines) 36
8.4 Analytics Tests (test_labels.py, 179 lines; test_model.py, 224 lines) 37
8.5 Performance Testing 38
8.6 Multi-Machine Testing 38
9. Problem Solving and Challenges 40
9.1 GPU Collector Bottleneck and GUI Freeze 40
9.2 CPU Frequency Reporting Inaccuracy 40
9.3 Database Session Overlap Bug 40
9.4 Deterministic Host UUID Design 41
9.5 GPU Absence Handling 41
9.6 pyqtgraph Graph Stutter 41
9.7 CPU Temperature - Deferred Feature 41
9.8 RAM Graph Hover Bug 42
9.9 GPU Library Change: GPUtil to pyNVML 42
10. Ethics 43
10.1 Data Privacy 43
10.2 AI Use Disclosure 43
10.3 System Monitoring Considerations 43
10.4 Accessibility and Plain-Language Design 43
11. Conclusion 45
12. Future Improvements 46
References 47
Appendix 49
Appendix A - AI Use Declaration 49
Appendix B - Full Label Threshold Specification 50
Appendix C - Software Dependencies 50

Summary

The Smart System Performance Dashboard is a Windows desktop application built entirely in Python that collects live CPU, RAM, GPU, and Disk telemetry every second, stores it in a local SQLite database, and uses a pre-trained Random Forest classifier to detect 12 categories of hardware fault, reporting findings in plain language. The system is structured as four independent services, Collector, Storage, Analytics, and Dashboard, running in parallel via background threads so the GUI stays fully responsive at all times.

The motivation for the project was a gap in existing tools. Applications like MSI Afterburner [19], HWMonitor [20], and Windows Task Manager display raw hardware numbers but offer no interpretation of what those numbers mean or whether anything is wrong. A user whose machine is slowing down due to GPU thermal throttling, a memory leak, or a failing drive has no straightforward way to identify the cause from existing tools alone. This application addresses that gap by combining real-time monitoring, persistent local storage, and automated fault detection in a single self-contained desktop tool that works fully offline with no account or internet connection required.

The Collector Service gathers hardware metrics every second using psutil, pywin32, and pynvml. All telemetry is written to a normalised 10-table SQLite schema with Write-Ahead Logging enabled, which allows the storage thread to write new samples at the same time as the analytics thread reads recent ones without either blocking the other. Every five seconds the Analytics Service reads the most recent 10 samples, extracts a 36-element feature vector, and runs it through a Random Forest classifier trained on over 20,000 samples. Detected faults are displayed in the PyQt6 dashboard with plain-language descriptions and suggested fixes, alongside a continuous health score between 0 and 100.

Collection cycles complete in 22 to 38ms, well within the 50ms target. The classifier achieves an average F1 score of 0.98 across all 12 fault categories. Over 80 automated tests across three suites verify every subsystem independently.

Poster

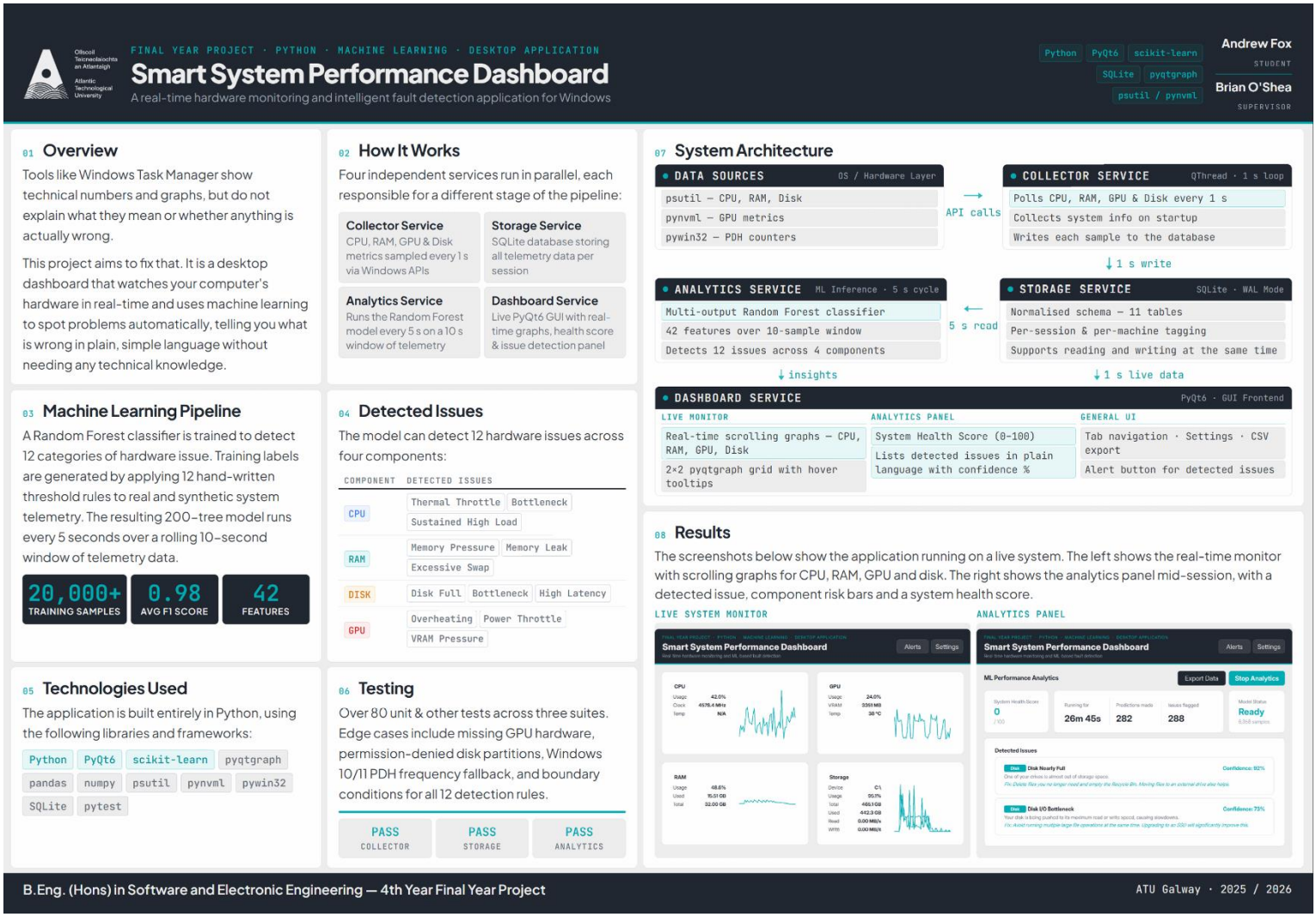


Figure: Project Poster

1. Introduction

Modern desktop computers, particularly those used for gaming, creative work, and software development, place heavy and variable demands on hardware. Tools like Windows Task Manager, MSI Afterburner [19], and HWMonitor [20] display technical numbers and graphs but do not explain what those numbers mean or whether anything is wrong. A user whose system is slowing down due to GPU thermal throttling, a memory leak, or a failing drive has no clear way to identify the cause from existing tools alone.

This project came about from a personal interest in both computer hardware and machine learning, and from the goal of building a tool that combined the two in a practical and useful way. The application was designed with two types of users in mind. The first is the technically experienced user who wants detailed, accurate hardware data and the ability to export that data for further analysis. The second is the everyday user who simply wants to know if something is wrong with their computer, without needing any technical knowledge to understand the answer. Existing tools serve the technically experienced user well. The everyday user, who simply wants a clear answer, is largely overlooked.

The Smart System Performance Dashboard addresses this gap. It is a Windows desktop application that monitors a computer's hardware in real-time, stores all telemetry locally, and graphs that data for the user. A trained machine learning model then analyses the stored data to detect hardware faults automatically, reporting its findings in plain, simple language and suggesting a course of action to resolve the issue.

The application is built around four independent services: the Collector, which gathers hardware metrics at one-second intervals using Windows system APIs; the Storage service, which persists all telemetry to a local SQLite database; the Analytics service, which runs a pre-trained Random Forest model against the stored user data; and the Dashboard, which presents everything to the user in real-time. The decision to target Windows was driven by the availability of Windows-specific APIs for hardware data collection and the practicality of testing on personal machines. Everything runs locally on the user's machine, with no cloud dependency of any kind. This was a deliberate design choice: the application works fully offline, requires no account or external service, and ensures that all telemetry data remains in the possession of the user. The full source code is publicly available at https://github.com/AndrewFoxATU/Final-Year-Project [15] This project demonstrates how a structured multi-service architecture can be used to collect, store, and intelligently analyse hardware data in a self-contained local application.

1.1 Report Structure

The remainder of this report is organised as follows. Section 2 provides background research and a review of existing tools. Section 3 presents the system architecture. Section 4 describes the project plan and development methodology. Section 5 covers the Live Monitoring subsystem. Section 6 covers the Database and Storage Service. Section 7 covers the Machine Learning and Analytics pipeline. Section 8 details the testing strategy and results. Section 9 documents key problems encountered and their

solutions. Section 10 addresses ethical considerations. Section 11 concludes the report and Section 12 outlines future improvements.

2. Background and Research

2.1 Existing System Monitoring Tools

Several tools exist for monitoring PC hardware performance, each serving a different audience and purpose.

MSI Afterburner [19] is widely used among gamers for its real-time GPU metrics and on-screen overlay. It displays clock speeds, temperatures, frame rates, and power draw, and allows users to log data to a file. However, it offers no diagnostic capability: it records numbers but draws no conclusions from them, and its logging format is not designed for programmatic analysis.

HWMonitor [20] and HWiNFO [21] both expose a comprehensive range of hardware sensor readings, including voltages, fan speeds, thermal values, and per-core CPU frequencies. These tools are respected among hardware enthusiasts and system builders for the depth of their data. They are not, however, designed for users who lack that background. Values are presented as raw numbers in dense tables, with no indication of whether a reading is normal, elevated, or dangerous.

NZXT CAM [22] takes a more consumer-oriented approach, presenting hardware data through a cleaner interface with basic health indicators. Its main limitation is that it requires a cloud account to function fully and sends telemetry data to NZXT's servers. For users who want their hardware data to remain private and entirely local, this is a significant drawback.

Windows Task Manager provides a real-time view of CPU, memory, disk, and network usage, and has improved substantially in recent Windows releases. It remains a general-purpose utility rather than a diagnostic one: it shows current load but offers no historical trend analysis, no fault detection, and no plain-language interpretation of what the readings indicate.

None of these tools combine persistent local storage of hardware telemetry, automated fault detection, and plain-language reporting in a single application. When a user experiencing intermittent slowdowns, unexpected shutdowns, or thermal issues they must either read raw sensor values and interpret them manually, or search for answers externally. The Smart System Performance Dashboard was built to fill this gap, targeting users at both ends of the experience spectrum.

2.2 Technology Selection and Justification

2.2.1 Python

Python was chosen as the primary language because it has strong library support for hardware monitoring, GUI development, and machine learning, covering everything this project needed in a single language. Python was also the natural choice given prior familiarity with the language and prior experience using it for machine learning through the Machine Learning with Python module, which was the first introduction to the topic.

2.2.2 PyQt6 and pyqtgraph

PyQt6 is a Python port for the Qt framework, I was first introduced to it during a work placement at Thermo King where it was used for internal monitoring dashboards and a telematics data visualisation tool. That experience showed it handles data-heavy, multi-threaded desktop applications well. Qt for Python documentation at https://doc.qt.io/qtforpython-6 covers the same API and was used as the primary reference throughout development. PyQt6 supports background threads well, which suited the Collector and Analytics services that run alongside the GUI without blocking the main thread.

2.2.3 psutil, pynvml, and pywin32

psutil (v6.0.0) was chosen for CPU, RAM, and disk data collection. It is a well-known and well-documented library that covers all of these metrics through a straightforward API, making it the natural starting point for hardware data collection in Python.

For GPU monitoring, GPUtil was used initially as it was the most referenced option in tutorials and documentation at the time. During testing it was found to be unreliable, with clock speed and temperature readings inconsistent across collection cycles. Investigation led to pynvml (nvidia-ml-py, v12.560.30), which is NVIDIA's own official Python interface to their NVML C library, the same library that powers the nvidia-smi command-line tool. The switch resolved the reliability issues and provided a broader set of metrics with lower overhead.

pywin32 (v307) was added during development after it became clear that psutil was not returning accurate CPU frequency on Windows. The Windows PDH (Performance Data Helper) counters exposed through pywin32 provide accurate real-time frequency readings including turbo boost, which psutil cannot obtain on Windows.

CPU temperature was investigated during Semester 1 as a potential metric. It was found that Windows provides no universal API for reading CPU temperature: different motherboard vendors expose thermal sensors in different ways, the WMI approach requires administrator privileges and does not work on all hardware, and the LibreHardwareMonitor approach requires a kernel driver. CPU temperature was deferred rather than shipping an implementation that would work on some machines and fail silently on others. This is discussed further in Section 9.7.

2.2.4 SQLite with WAL Mode

SQLite was selected as the storage engine because it is serverless and self-contained, with the only file dependency being the .db file itself, requiring no separate database installation. Write-Ahead Logging (WAL) mode was enabled via PRAGMA journal_mode = WAL, which allows the Storage Service to write new samples at 1 Hz while the Analytics Service reads the most recent 10 samples at 0.2 Hz, with no read/write contention. PRAGMA synchronous = NORMAL balances write durability with performance, and PRAGMA busy_timeout = 5000 prevents blocking errors when both threads access the database simultaneously.

2.2.5 scikit-learn Random Forest

Three candidate approaches were considered for the fault detection component. K-Means clustering was the first evaluated, having been covered during the Machine Learning with Python module. It is an unsupervised method that groups similar data points together, but it cannot assign named labels to those groups. It would identify that a hardware reading is unusual without being able to say what kind of fault it represents, which was not sufficient for the requirements.

Isolation Forest was considered next. It is a well-regarded anomaly detection algorithm that identifies samples which differ significantly from normal behaviour, and it is well-suited to tabular sensor data. Its limitation is that it produces a binary output: normal or anomalous. It cannot distinguish between a GPU thermal throttle event and a memory leak. For an application that needs to tell a user specifically what is wrong with their machine, a binary flag is not useful.

Random Forest was selected because it is a supervised learning algorithm that supports labelled classification directly. It works by building a large number of decision trees during training, each trained on labelled examples of what a fault looks like, with each tree voting on the outcome and the majority result taken. This fitted well with the rule-based labelling approach used in this project, where training labels are generated automatically by the LabelEngine rather than being labelled by hand. It works well with tabular hardware data, produces a probability score alongside each prediction rather than just a yes or no, and does not need much tuning to get good results. The probability scores were particularly useful for this project as they are used to display a confidence percentage alongside each detected issue in the Analytics Panel.

Because 12 labels need to be predicted at the same time, scikit-learn's MultiOutputClassifier was used to wrap the base estimator, training one Random Forest per label in parallel.

2.3 Requirements

The following functional requirements were established based on background research and the project proposal:

- FR1: The system shall collect CPU, RAM, GPU, and Disk metrics every 1 second via Windows APIs.
- FR2: The system shall persist all collected metrics to a local SQLite database, organised across 10 normalised tables with per-host and per-session tagging.
- FR3: The system shall run the pre-trained Random Forest classifier every 5 seconds on a rolling 10-second window of telemetry and display detected faults with confidence percentages.
- FR4: The system shall display a real-time system health score between 0 and 100, calculated by summing a confidence-scaled penalty for each detected fault label, where each label's penalty is determined by its severity rating (1-5) mapped to a penalty value (2, 5, 15, 35, or 60 points) and multiplied by the model's confidence probability for that label, subtracted from 100 and clamped to the valid range.

- FR5: The user shall be able to configure the graph refresh rate (100-10,000 ms) and UI accent colour, persisted to ~/.smart_dashboard_settings.json across sessions.
- FR6: The system shall provide CSV export of analytics results.
- FR7: The system shall handle machines without a discrete NVIDIA GPU without crashing or degrading other functionality.

Note: the original project proposal included two additional planned features, a RAM Cleaner and a Performance Mode, both listed as if-time-allows additions from the start. During development the focus was kept on getting the four core services right rather than rushing to add extra features, so both were dropped. They could be revisited as future additions but are not part of the submitted project.

Non-functional requirements:

- NFR1: Each data collection cycle shall complete within 50ms to prevent GUI freezing.
- NFR2: The application shall run on Windows 10 and Windows 11 without administrator privileges.
- NFR3: The application shall add no more than 5% CPU overhead during normal monitoring.

Success criteria (as defined in the winter presentation and verified at the end of each semester):

- SC1 (Semester 1): Graph updates are smooth with no UI freezing. Verified: collection cycles run in 22-38ms; GUI thread is never blocked.
- SC2 (Semester 1): CPU, RAM, GPU, and Disk metrics align with values shown in Windows Task Manager. Verified: manual comparison confirmed values match within expected tolerances, accounting for the fact that Task Manager and the application poll hardware at different times and will not produce identical readings at the same instant.
- SC3 (Semester 2): Historical telemetry data is stored and queryable from the SQLite database, and the user can export the full database to a timestamped CSV file. Verified: StorageManager correctly persists all sample types; get_recent_samples() returns time-ordered rows; Export DB produces a valid CSV containing host info, session data, and all per-second samples.
- SC4 (Semester 2): The ML model outputs detected issue labels with confidence scores and a continuous health score between 0 and 100. Verified: PerformanceModel.predict() returns issues list with probabilities and health score; all 12 labels fire correctly in test_labels.py and test_model.py.
- SC5 (Semester 2): The application runs correctly on Windows 10 and Windows 11, with and without a NVIDIA GPU. Verified: tested across four physical machines covering both operating systems and both GPU configurations, with correct behaviour confirmed in each case.

3. System Architecture

The Smart System Performance Dashboard is structured as four independent services running in parallel, each responsible for a distinct stage of the data pipeline. Each service can be developed, tested, and profiled independently.

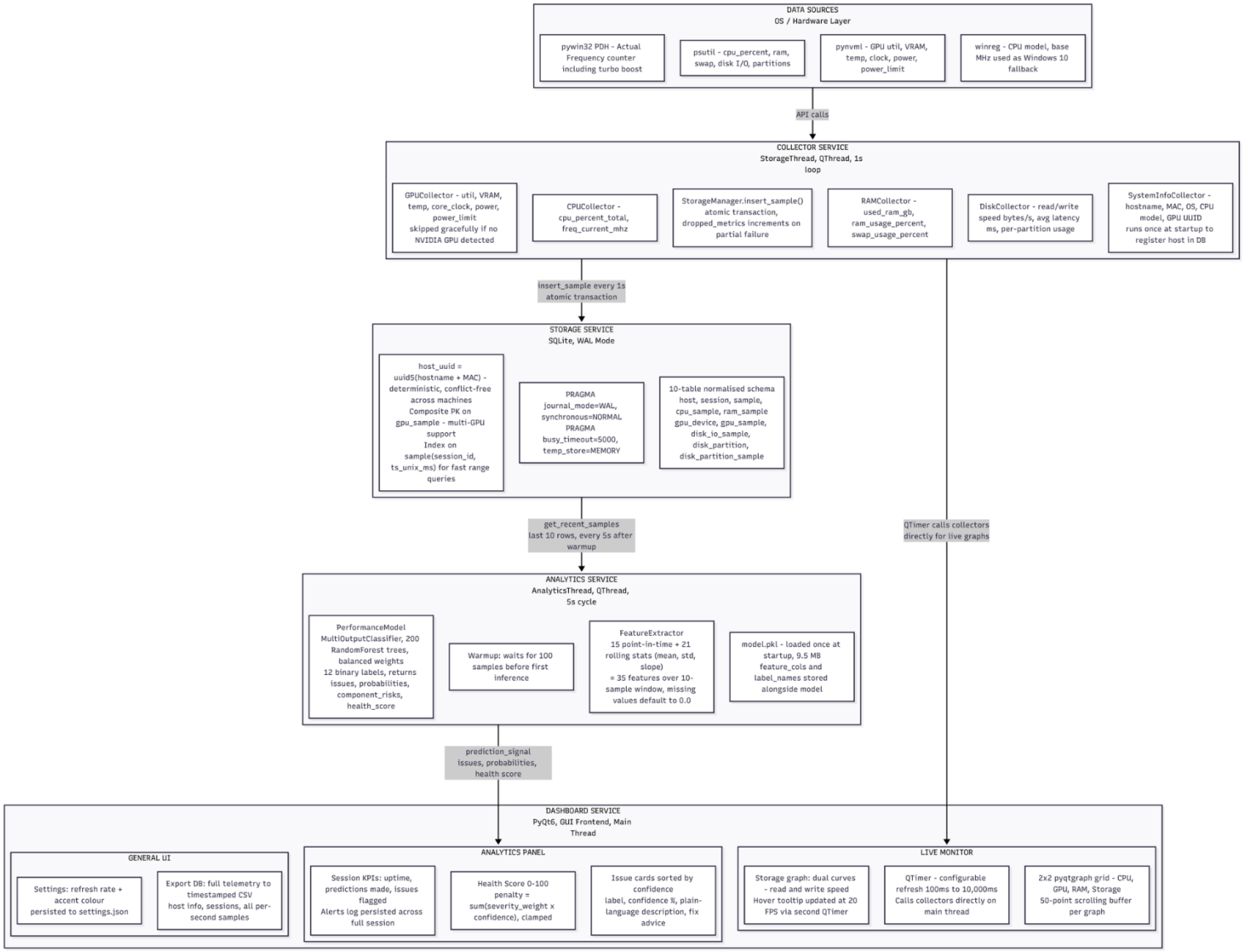


Figure: System Architecture Diagram, 4-service pipeline (Collector, Storage, Analytics, Dashboard)

3.1 The Four Services

3.1.1 Collector Service (QThread, 1-second loop)

The Collector Service runs inside a background QThread and polls CPU, RAM, GPU, and Disk hardware every second using Windows APIs. CPUCollector uses psutil for utilisation and pywin32 PDH counters for accurate real-time frequency including turbo boost. RAMCollector reads RAM and swap usage via psutil. GPUCollector gathers utilisation, VRAM, temperature, clock speed, and power data via pynvml, and is skipped gracefully if no NVIDIA GPU is present. DiskCollector measures read and write speed in bytes per second, average read and write latency in milliseconds, and

per-partition usage. On startup, SystemInfoCollector runs once to register the host machine in the database, capturing hostname, MAC address, OS version, CPU model, and GPU UUID.

Each collection cycle is timed and the duration stored as collect_duration_ms. If any individual collector fails, the others continue and a dropped_metrics counter is incremented for that sample. The entire sample is written as a single database transaction, so if the write fails, nothing is partially saved.

3.1.2 Storage Service (SQLite, WAL Mode)

The Storage Service receives samples from the Collector and writes them to a 10-table normalised SQLite schema. Write-Ahead Logging mode is enabled so the Analytics Service can read from the database at the same time as the Collector is writing to it, without either thread blocking the other. PRAGMA synchronous = NORMAL and PRAGMA busy_timeout = 5000 are set for performance and reliability.

Each machine is identified by a deterministic UUID generated as uuid5(hostname + MAC address), so a given machine always produces the same identifier regardless of when or where the database is run. This makes it possible to merge database files from different machines without primary key conflicts, which was used during training data collection. All data is tagged per-host and per-session, with a composite primary key on gpu_sample to support machines with multiple GPUs.

3.1.3 Analytics Service (ML Inference, 5-second cycle)

The Analytics Service runs in a second background QThread. It waits until 100 samples have been collected, roughly one and a half minutes of data, before running its first prediction. This ensures the rolling-window statistical features are computed over a full window of data rather than just a handful of samples.

Once the warmup period is complete, every five seconds the service reads the 10 most recent samples from the database, passes them through FeatureExtractor to produce a 36-element feature vector, and runs inference using PerformanceModel. The model returns a list of detected issues, a confidence probability for each label, and a health score between 0 and 100. These results are emitted to the Dashboard via a Qt thread signal, where each detected issue is displayed with a plain-language description of the problem and a suggested fix.

3.1.4 Dashboard Service (PyQt6 GUI Frontend)

The application opens directly to the Live Monitor tab on startup with no loading screen. The Live Monitor displays a 2x2 grid of pyqtgraph scrolling graphs covering CPU, GPU, RAM, and Storage, each with live metric labels and a hover tooltip updated at 20 FPS. The graph refresh rate is configurable from 100ms to 10,000ms. On machines with no NVIDIA GPU the GPU panel is left blank and all GPU-dependent functionality is skipped without errors.

The Analytics Panel shows the current health score, model status, session statistics, and a scrollable list of issue cards. Each card displays the fault name, confidence percentage, a plain-language description of the problem, and a suggested fix. The panel shows a collecting indicator until the 100-sample warmup is complete. An alerts button in the top bar opens a log of all issues detected across the full session. The Export Analytics button opens a dialog where the user can choose to export a session summary, the current detected issues list, or both, saved as a timestamped CSV file.

The top bar also includes an Export DB button which exports the full telemetry database to a CSV file. This includes host information, all sessions, and every per-second sample collected, giving technically experienced users access to their raw hardware data for further analysis outside the application.

3.2 Key Design Decisions

- The decision to run the Collector and Analytics services as separate QThreads, independent of each other and of the GUI main thread, was the most important structural choice in the project. It means no data collection or inference work can block the user interface. The GUI timer fires on its own schedule entirely separately from the two background threads.
- Each service is an independent Python module with its own test suite, making it straightforward to profile, debug, and test each part of the system in isolation. This separation proved valuable during development when the GPU collector was identified as the cause of a 0.5 to 1 second GUI freeze on every collection cycle. Because the collector service was fully separate, it could be profiled and timed in isolation using a dedicated debug script, the root cause was identified and fixed, and no other part of the codebase needed to change. This is covered in detail in Section 9.1.
- SQLite with WAL mode was chosen specifically because it allows the Storage Service to write new samples at 1 Hz at the same time as the Analytics Service reads the last 10 samples at 0.2 Hz, with no contention between them. A traditional SQLite journal mode would have caused one thread to block the other.
- The model is loaded once from model.pkl at application startup and kept in memory for the lifetime of the session. This avoids any retraining or loading overhead during inference and keeps each 5-second cycle fast. The feature column names and label names are stored inside model.pkl alongside the model object itself, so the inference code never needs to hardcode the feature order.
- Each machine is assigned a deterministic UUID generated as uuid5(hostname + MAC address). This is covered in detail in Section 3.1.2 but is worth noting as a key design decision, as it was what made merging database files from multiple machines possible during training data collection.

4. Project Plan and Development Methodology

4.1 Agile Scrum Methodology

The project was managed using Agile Scrum across both semesters. Weekly stand-up meetings were held as part of the Project Engineering module and progress was tracked in Jira alongside weekly notes in OneNote. Figure 4.3 shows an example of the weekly stand-up notes recorded in OneNote throughout the project. Each entry documents what was worked on that week, any blockers that came up, and what was planned for the following week. Tasks were created at the start of each sprint, assigned to the current sprint, and marked as done as work was completed. The Jira board used the AFFYP- ticket prefix and a timeline view to track progress visually across the full year, as shown in Figure 4.1.

Having an active backlog made it easier to handle blockers when they came up. The biggest example of this was in Semester 2 Week 4, when a 0.5 to 1 second GUI freeze was discovered while the storage service was being built. Rather than pushing on with the planned database work, the sprint was reprioritised on the spot to do a proper root cause analysis first. That turned out to be the right call as the fix had a direct impact on everything that came after. Being on a solo project meant there was no team to negotiate reprioritisation with, so this kind of flexibility was one of the most useful things about using Scrum.

4.2 Version Control

All source code was managed in a GitHub repository at https://github.com/AndrewFoxATU/Final-Year-Project using a feature-branch workflow. Each piece of work was done on its own branch and merged into master when finished. Pull requests were used when switching between my desktop PC and laptop, so changes were always reviewed and cleanly merged before picking up on the other machine. A GitHub merge conflict in Semester 1 Week 5 was the first real test of this workflow and it was resolved without any lost work. Figure 4.2 shows the commit history across both semesters. The project built up over 50 commits with a steady and consistent history throughout, demonstrating that development was incremental and ongoing rather than produced in a single session.

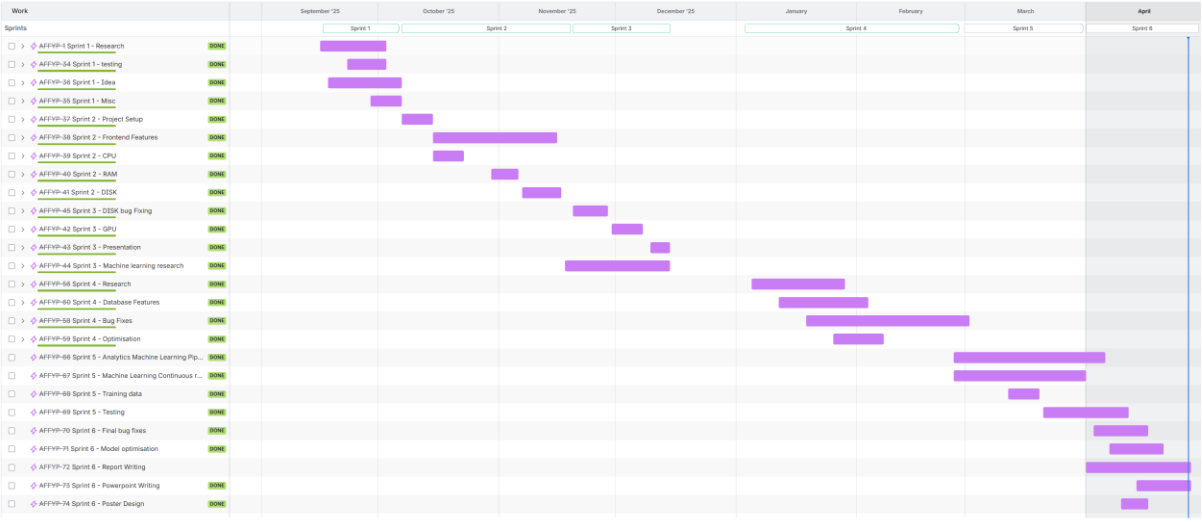
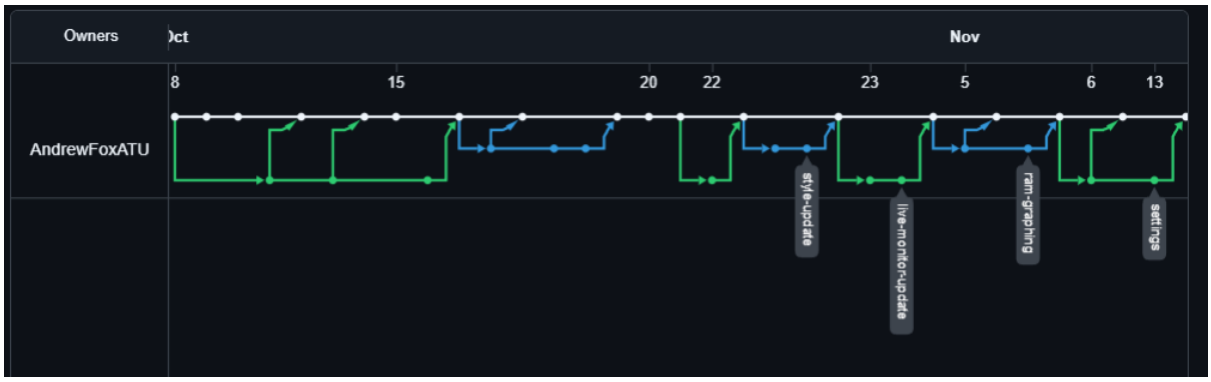


Figure 4.1 - Jira timeline view showing all six sprints across both semesters, with AFFYP- tickets mapped against their completion dates.



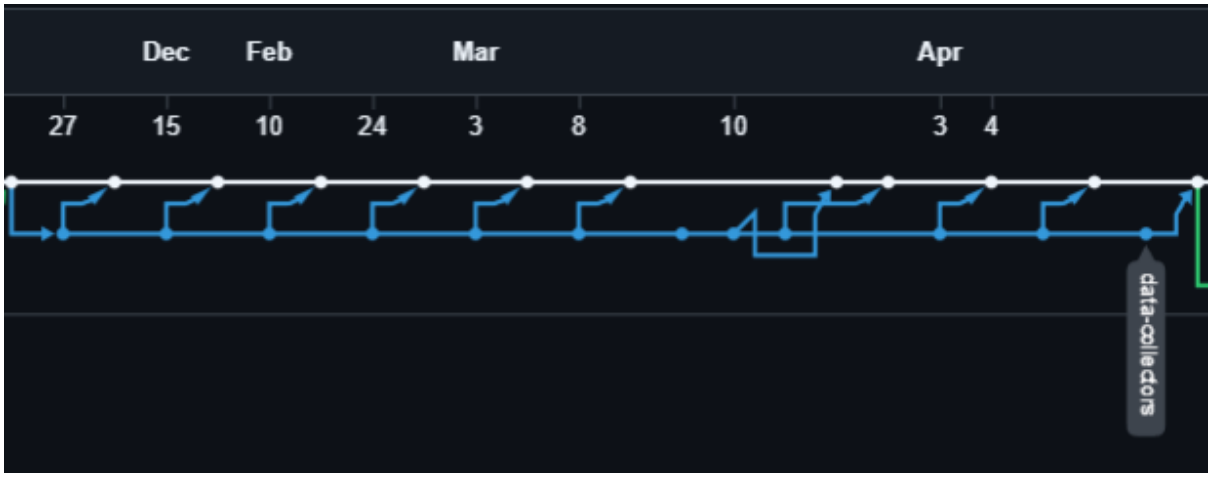
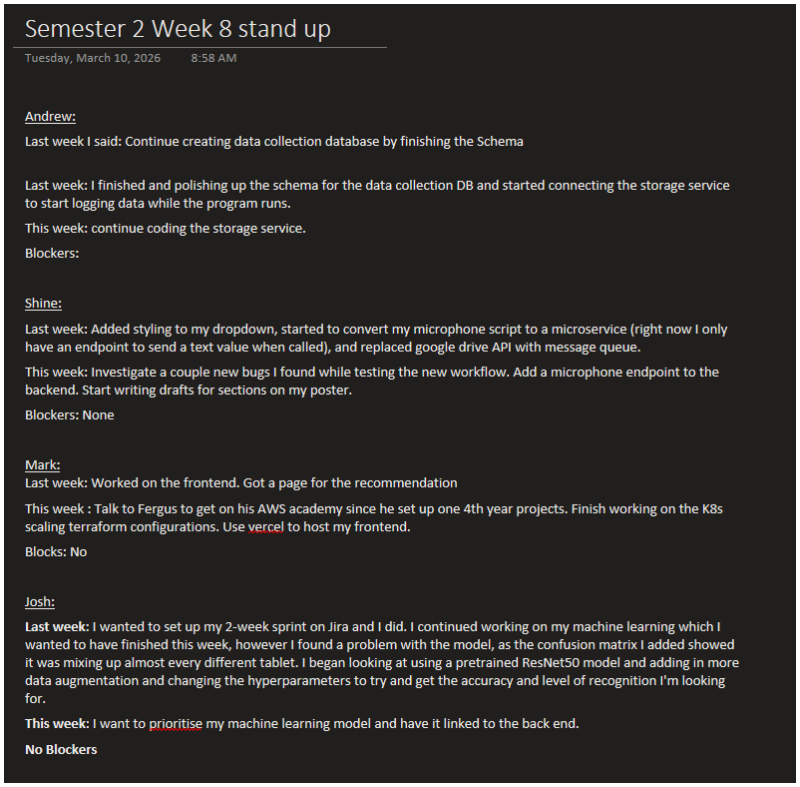


Figure 4.2 - GitHub commit history showing a steady and incremental development timeline across Semester 1 and Semester 2.



Semester 2 Week 8 stand up

Tuesday, March 10, 2026 8:58 AM

Andrew:

Last week I said: Continue creating data collection database by finishing the Schema

Last week: I finished and polishing up the schema for the data collection DB and started connecting the storage service to start logging data while the program runs.

This week: continue coding the storage service.

Blockers:

Shine:

Last week: Added styling to my dropdown, started to convert my microphone script to a microservice (right now I only have an endpoint to send a text value when called), and replaced google drive API with message queue.

This week: Investigate a couple new bugs I found while testing the new workflow. Add a microphone endpoint to the backend. Start writing drafts for sections on my poster.

Blockers: None

Mark:

Last week: Worked on the frontend. Got a page for the recommendation

This week : Talk to Fergus to get on his AWS academy since he set up one 4th year projects. Finish working on the K8s scaling terraform configurations. Use vercel to host my frontend.

Blocks: No

Josh:

Last week: I wanted to set up my 2-week sprint on Jira and I did. I continued working on my machine learning which I wanted to have finished this week, however I found a problem with the model, as the confusion matrix I added showed it was mixing up almost every different tablet. I began looking at using a pretrained ResNet50 model and adding in more data augmentation and changing the hyperparameters to try and get the accuracy and level of recognition I'm looking for.

This week: I want to prioritise my machine learning model and have it linked to the back end.

No Blockers

Figure 4.3 - Weekly stand-up notes recorded in OneNote throughout the project, documenting progress, blockers, and decisions made each week.

5. Live System Monitoring

The Live Monitoring module is the primary visible feature of the dashboard. It provides real-time hardware metrics updated every second and rendered as a 2x2 grid of scrolling pyqtgraph plots with live hover tooltips.

5.1 Data Collection Backend

The collector service is made up of four classes, one for each hardware component: CPUCollector, RAMCollector, GPUCollector, and DiskCollector. A fifth class, SystemInfoCollector, runs once at startup to record static host information such as hostname, operating system, CPU model, core count, and total RAM. This data is stored in the host table and does not need to be collected again during the session.

Each collector exposes a static get_*_data() method that is called once per second by StorageThread running in the background and also called by LiveSystemMonitor on its own timer to update the graphs. The collectors do not hold state between calls, they just read the current hardware values and return them as a dictionary.

RAMCollector is the simplest of the four. It uses psutil to read total RAM, used RAM in gigabytes, RAM usage as a percentage, and swap usage as a percentage. These are straightforward psutil calls with no platform-specific logic needed.

CPUCollector is more complex because psutil does not reliably report current CPU frequency on Windows. To get an accurate reading, the collector uses the Windows PDH (Performance Data Helper) API via pywin32. On Windows 11, it queries the \Processor Information(_Total)\Actual Frequency counter, which gives the real operating frequency in megahertz. On Windows 10, that counter is not available, so the collector falls back to reading \Processor Information(_Total)\% Processor Performance, which gives a percentage of the base clock, and multiplies it by the base frequency taken from psutil. CPU load percentage is read using psutil.cpu_percent(interval=None), which returns the value since the last call without blocking.

DiskCollector calculates read and write speeds by comparing the current I/O counters to the previous call. It divides the difference in bytes transferred by the time elapsed since the last sample to get a bytes-per-second figure. Latency is calculated similarly, by dividing the change in total read or write time by the change in read or write operation count. Partition usage is collected by calling psutil.disk_partitions() and reading total, used, and usage percentage for each. If a partition raises a PermissionError during the usage check, it is skipped rather than crashing the collector.

GPUCollector uses the pynvml library to query NVIDIA GPU metrics including utilisation, VRAM used, temperature, core clock speed, and power draw. When the module is imported, it attempts to initialise pynvml and sets a module-level gpu_available flag to True or False. If no NVIDIA GPU is present, or if initialisation fails, gpu_available is set to False and the GPU collector is skipped entirely for the rest of the session. The GPU panel in the dashboard is simply left blank rather than showing placeholder values.

5.2 Performance Optimisation

During Semester 2, a performance issue was identified where the GUI was freezing for roughly 0.5 to 1.0 seconds on every collection cycle. A structured five-step root cause analysis was carried out to find and fix it.

Step 1 - Define the Problem. The GUI was freezing for 0.5 to 1.0 seconds each time the collectors ran, making the dashboard feel unresponsive during live monitoring.

Step 2 - Collect Data. Custom profiling code was written to time each collector individually over ten iterations, and then all four together. CPU averaged 1.94ms, Disk averaged 0.92ms, and RAM averaged around 7ms in steady state. GPU was the clear outlier, ranging from 112ms to over 1,200ms per call and averaging around 344ms. Full collection cycles were regularly taking 300 to 700ms, with spikes above 1,000ms.

Step 3 - Identify Causal Factors. The GPU collector was immediately the primary suspect given how far it was above the others. Disabling it stopped the freezing entirely, which confirmed it was the source.

Step 4 - Root Cause. Two issues were found inside gpu_collector.py. The PCIe throughput metrics, gpu_pcie_tx_mb and gpu_pcie_rx_mb, were significantly slower than all other GPU queries. On top of that, the pynvml session was being initialised and shut down on every single collection call, adding unnecessary overhead each time.

Step 5 - Solution. The PCIe metrics were removed as they were not used anywhere in the feature extractor or the dashboard. The pynvml session was moved to module-level initialisation so it only opens once at startup. After both changes, the GPU collector dropped from an average of 344ms to around 22ms. Full collection cycles now complete in 22 to 38ms, well within the 50ms non-functional requirement.

```
=== Testing all together ===
Iteration 1: CPU=0.66ms, RAM=0.97ms, GPU=681.50ms, Disk=1.22ms, TOTAL=684.36ms
Iteration 2: CPU=0.86ms, RAM=1.08ms, GPU=268.33ms, Disk=1.31ms, TOTAL=271.59ms
Iteration 3: CPU=4.38ms, RAM=2.74ms, GPU=706.41ms, Disk=1.15ms, TOTAL=714.69ms
Iteration 4: CPU=0.67ms, RAM=1.26ms, GPU=342.97ms, Disk=1.29ms, TOTAL=346.19ms
Iteration 5: CPU=1.44ms, RAM=2.70ms, GPU=112.93ms, Disk=1.47ms, TOTAL=118.56ms
Iteration 6: CPU=0.86ms, RAM=2.72ms, GPU=732.72ms, Disk=1.30ms, TOTAL=737.61ms
Iteration 7: CPU=1.30ms, RAM=2.89ms, GPU=436.99ms, Disk=0.45ms, TOTAL=441.63ms
Iteration 8: CPU=1.61ms, RAM=2.69ms, GPU=339.93ms, Disk=1.52ms, TOTAL=345.77ms
Iteration 9: CPU=0.43ms, RAM=1.42ms, GPU=1205.32ms, Disk=1.23ms, TOTAL=1208.41ms
Iteration 10: CPU=0.68ms, RAM=1.56ms, GPU=235.31ms, Disk=1.32ms, TOTAL=238.87ms
```

```
=== Testing all together ===
Iteration 1: CPU=0.27ms, RAM=7.57ms, GPU=22.70ms, Disk=5.63ms, TOTAL=36.16ms
Iteration 2: CPU=0.09ms, RAM=7.40ms, GPU=10.37ms, Disk=5.72ms, TOTAL=23.58ms
Iteration 3: CPU=0.09ms, RAM=7.42ms, GPU=11.38ms, Disk=5.89ms, TOTAL=24.78ms
Iteration 4: CPU=0.16ms, RAM=7.35ms, GPU=23.62ms, Disk=6.02ms, TOTAL=37.15ms
Iteration 5: CPU=0.23ms, RAM=7.79ms, GPU=24.87ms, Disk=5.78ms, TOTAL=38.67ms
Iteration 6: CPU=0.23ms, RAM=7.71ms, GPU=8.92ms, Disk=5.24ms, TOTAL=22.11ms
Iteration 7: CPU=0.13ms, RAM=7.34ms, GPU=23.33ms, Disk=5.27ms, TOTAL=36.07ms
Iteration 8: CPU=0.21ms, RAM=7.31ms, GPU=9.88ms, Disk=5.72ms, TOTAL=23.13ms
Iteration 9: CPU=0.22ms, RAM=7.38ms, GPU=23.86ms, Disk=5.24ms, TOTAL=36.70ms
Iteration 10: CPU=0.38ms, RAM=7.76ms, GPU=11.92ms, Disk=5.30ms, TOTAL=25.36ms
```

Figure: Collector profiling results before and after optimisation

5.3 GUI Implementation

The main window is built in DashboardWindow and is divided into three areas: a top bar, a swappable content area, and a bottom navigation bar. The top bar contains the application title and three buttons: Alerts, Export DB, and Settings. The bottom bar has two navigation buttons, Live System Monitoring and Analytics, which swap the visible panel in the content area by hiding the inactive widget and showing the selected one. This is a straightforward show/hide approach rather than a tab widget, which gave more control over layout and styling, and also means both panels continue running in the background while hidden, so the analytics thread keeps processing and the graphs keep collecting data even when the user is not on that tab.

The Live System Monitoring panel is a 2x2 grid containing four section frames, one each for CPU, GPU, RAM, and Storage. Each frame holds a pyqtgraph PlotWidget on the right and a column of metric labels on the left showing the current values. The graphs display the last 50 data points as a scrolling line, updated by a QTimer at the configured refresh rate. A second timer running at 50ms handles hover detection independently, so the hover tooltip updates smoothly at around 20 frames per second regardless of the graph refresh rate.

Styling is handled through a QSS stylesheet at /assets/styles/style.qss. The stylesheet contains a placeholder string ACCENT_COLOUR which is replaced with the actual hex colour value at startup before the stylesheet is applied to the application. This means the accent colour is applied globally across all buttons, graph lines, and UI elements from a single value, without any conditional logic in the widget code.

The Inter font is loaded at startup using QFontDatabase.addApplicationFont for three weights: Regular, Bold, and ExtraBold. Once loaded, the font family name is retrieved and set as the application-wide font. All text in the dashboard inherits it from there, with specific weights applied through the QSS stylesheet using font-weight rules.

Settings are managed by settings_manager.py, which reads from and writes to a JSON file at ~/.smart_dashboard_settings.json. The two configurable values are the graph refresh rate in milliseconds and the accent colour as a hex string, with defaults of 1000ms and red. The Settings modal is a QDialog with a spin box and a hex input field. The refresh rate applies immediately to the live monitor timer, but the accent colour requires a restart since the QSS is only processed once at startup.

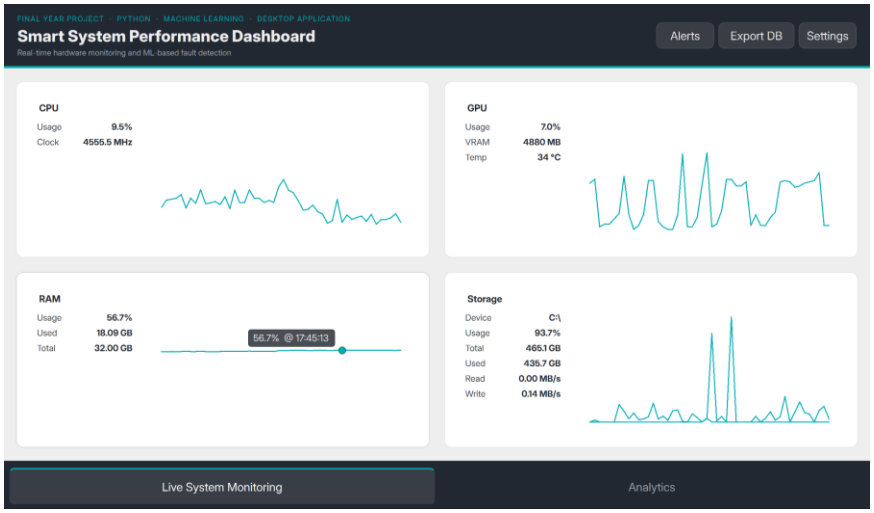


Figure: Dashboard GUI - Live Monitor tab showing CPU, GPU, RAM, and Storage graphs

5.4 Real-Time Graphing

Matplotlib was the first option looked at for live graphing, but it was not suitable for updating at one second intervals. It re-renders the entire figure on each update, which caused noticeable lag and made the dashboard feel unresponsive. pyqtgraph was chosen instead because it uses Qt's graphics scene for rendering, which is much faster and handles frequent updates without any visible stuttering.

Figure 5.4.1 shows the very first prototype built to test whether pyqtgraph could handle live hardware data. At this stage the application was just a single window with two tab buttons for CPU and RAM, and the graph used a filled area style. The hover tooltip showing the value and timestamp was already working at this point, which confirmed that pyqtgraph was the right choice before any further work was done on the layout.

Figure 5.4.2 shows an earlier version of the full dashboard layout, before the final styling was applied. The 2x2 grid of hardware panels was already in place, along with the bottom navigation bar. At this stage the dashboard had four navigation options including RAM Cleaner and Performance Mode, which were later cut to keep the scope focused on the core monitoring and analytics functionality.

Each PlotWidget keeps a rolling buffer of up to 50 data points. On each update, the new value is appended and the oldest is dropped, so the graph scrolls continuously from right to left. The graph update rate is configurable through the Settings panel between 100ms and 10,000ms, with a default of 1,000ms, and is saved to ~/.smart_dashboard_settings.json so it persists across sessions. Hover support is handled by a separate timer running at 50ms, which finds the nearest data point to the cursor and displays a label showing the value and timestamp.

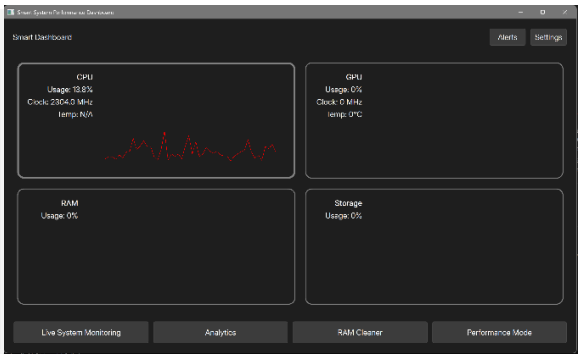


Figure 5.4.1 (left) - First pyqtgraph prototype, testing live CPU usage graphing with hover tooltip before the full dashboard layout was built.

Figure 5.4.2 (right) - Early version of the Live System Monitoring panel showing the 2x2 grid layout, before RAM Cleaner and Performance Mode were removed from scope.

6. Database and Storage Service

6.1 Design Rationale

SQLite was chosen as the storage engine because it is serverless and self-contained. There is no separate database process to install or manage, the entire database is a single .db file sitting alongside the application. All telemetry stays on the user's machine with no network access of any kind.

Four PRAGMAs are set on every connection. PRAGMA foreign_keys = ON enforces referential integrity across the schema. PRAGMA journal_mode = WAL enables Write-Ahead Logging, which allows the storage thread to write new samples at 1Hz at the same time as the analytics thread reads recent samples at 0.2Hz, without the two blocking each other. PRAGMA synchronous = NORMAL reduces the number of fsync calls compared to the default FULL setting, which improves write performance while still being safe enough for local telemetry data. PRAGMA busy_timeout = 5000 means that if a connection does briefly have to wait for the database, it will retry for up to five seconds before giving up rather than failing immediately.

The database file is named telemetry.db and is created in the directory the application is launched from. A user running the app from a project folder will find the file there. This keeps it easy to locate without any configuration.

6.2 Schema Design

The schema is normalised across 10 tables to avoid repeating metadata in every row and to keep large amounts of telemetry efficient to query. The schema diagram from DBVisualiser is shown in Figure 6.1.

Table	Primary Key	Purpose
host	host_uuid (TEXT)	Static machine details, registered once. UUID generated from hostname and MAC address.
session	session_id (INTEGER)	One row per application run. Foreign key: host_uuid.
sample	sample_id (INTEGER)	One row per 1-second tick. Foreign key: session_id. Stores collect_duration_ms and dropped_metrics.
cpu_sample	sample_id (foreign key)	cpu_percent_total and freq_current_mhz for the tick.
ram_sample	sample_id (foreign key)	used_ram_gb, ram_usage_percent, swap_usage_percent.
gpu_device	gpu_uuid (TEXT)	GPU registry. One row per GPU per host. Foreign key: host_uuid.

gpu_sample	(sample_id, gpu_uuid)	Per-GPU metrics per tick. Composite primary key supports multi-GPU machines.
disk_io_sample	sample_id (foreign key)	System-wide read/write speeds in bytes per second and latency in milliseconds.
disk_partition	partition_id (INTEGER)	Partition registry. Unique on (host_uuid, device, mountpoint). Foreign key: host_uuid.
disk_partition_sample	(sample_id, partition_id)	Per-partition usage (total_gb, used_gb, usage_percent) per tick.

The sample table has a composite index on (session_id, ts_unix_ms) for fast time-range queries. The disk_partition table has a UNIQUE constraint on (host_uuid, device, mountpoint) to prevent duplicate partition registrations across sessions. All foreign keys use cascading deletes, so removing a sample automatically cleans up all associated component rows.

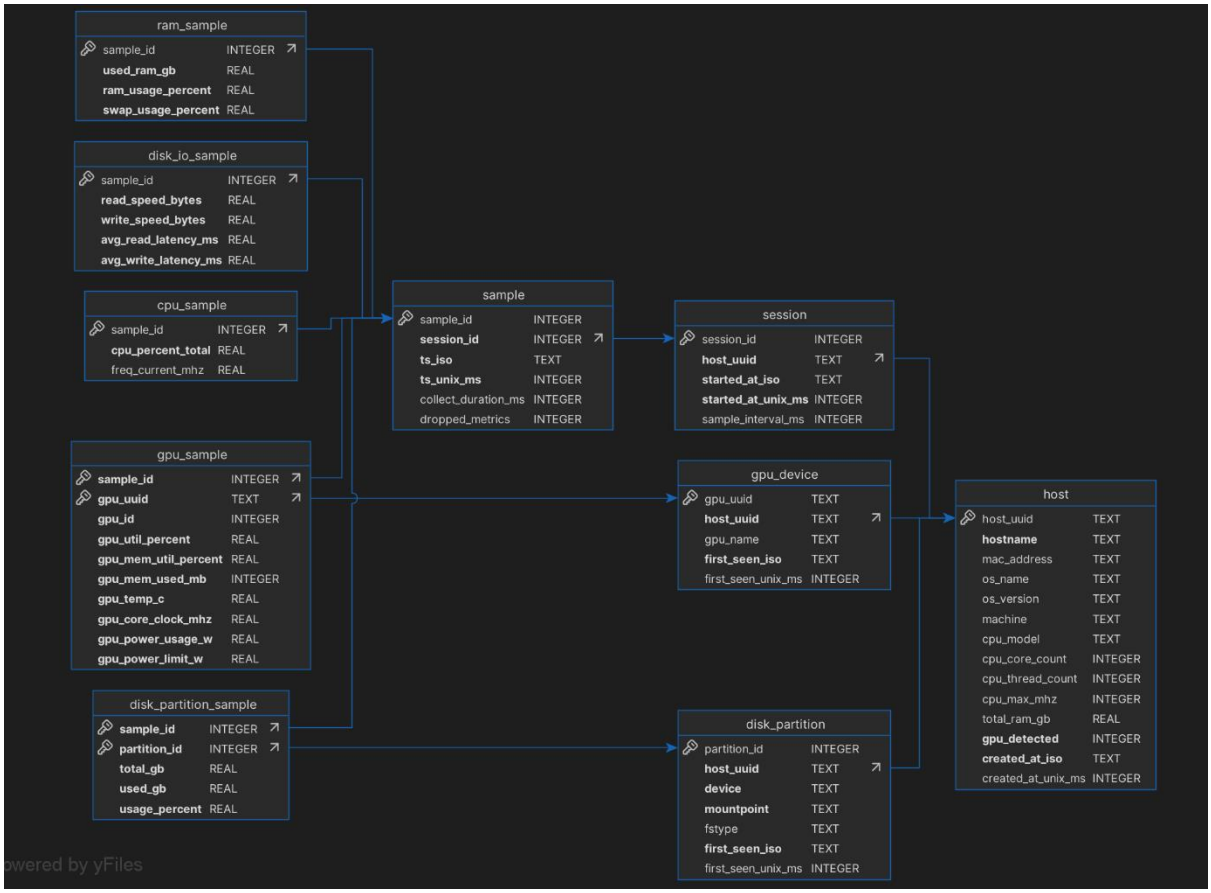


Figure 6.1 - Schema diagram showing all 10 tables and their relationships. (DBVisualiser)

6.3 Storage Service Implementation

StorageManager is the class that handles all database interaction, defined in storage_service/storage/main.py and running inside StorageThread in the background. On construction it initialises the schema, registers the host using INSERT OR IGNORE so the same machine is never duplicated across sessions, and registers any GPU devices the same way. It then inserts a new session row and stores the session_id for use in all subsequent inserts. GPU UUIDs and partition IDs are cached in dictionaries after first registration to avoid repeated database lookups on every tick.

insert_sample() is called once per second and writes all component data for that tick in a single transaction. Each component insert, CPU, RAM, GPU, disk I/O, and each disk partition, is wrapped in its own try-except block inside the transaction. If one component fails, dropped_metrics increments and the others continue as normal. This means a single faulty collector does not cause an entire second of data to be lost. At the end of the transaction, if any components were dropped, the sample row is updated with the final count. If the overall transaction itself fails, a ROLLBACK is issued so no partial data is written.

6.4 Training Data Collection

The storage system was also used to collect real-world training data. The collect_real_data.py script in the analytics service reads sessions from existing .db files and exports the raw telemetry as rows in the training CSV. Data was collected from three machines: my laptop, my desktop PC, and a friend's PC, covering Windows 10 and Windows 11, with and without a discrete NVIDIA GPU. This produced approximately 12,000+ rows of real telemetry, which was combined with approximately 8,000+ rows of synthetic data to form the full 20,000+ sample training set. A source column in the CSV distinguishes real from synthetic rows.

7. Machine Learning and Analytics

7.1 Model Selection

Model selection was covered in Section 2.2. Random Forest was chosen over K-Means and Isolation Forest because it supports named multi-label classification and produces per-label confidence probabilities. The specific configuration and results are covered in Section 7.6.

7.2 The 12 Detected Fault Categories

The 12 fault labels are defined in labels.py. Each one is a boolean condition applied to the feature dict produced by FeatureExtractor.compute(). The thresholds were defined manually based on hardware knowledge and online research into typical performance limits.

Component	Label	Trigger Condition
CPU	cpu_thermal_throttle	cpu_percent > 70 and frequency rolling slope < -50 MHz per sample
CPU	cpu_bottleneck	cpu_percent > 90 and ram_percent < 70 and disk_usage < 50
CPU	cpu_sustained_high_load	Rolling mean of cpu_percent > 88 across the window
RAM	ram_pressure	ram_percent > 80 and cpu_percent < 50
RAM	ram_memory_leak	RAM rolling slope > 0.3 and swap rolling slope > 0.05
RAM	excessive_swap_usage	swap_percent > 50
Disk	disk_full	disk_usage_percent > 90
Disk	disk_bottleneck	cpu_percent < 40 and ram_percent < 60 and read or write speed > 200 MB/s
Disk	disk_high_latency	avg_read_latency_ms > 20 or avg_write_latency_ms > 20
GPU	gpu_overheating	gpu_temp_c > 85
GPU	gpu_power_throttle	Power usage >= 98% of power limit and clock rolling slope < -5 MHz per sample
GPU	gpu_vram_pressure	gpu_mem_util_percent > 90

7.3 Feature Engineering

Each inference cycle operates on a rolling 10-sample window covering the last 10 seconds of data. From these 10 samples, FeatureExtractor.compute() builds a flat feature vector of 36 values.

15 of those are point-in-time values taken from the most recent sample: cpu_percent_total, freq_current_mhz, ram_usage_percent, swap_usage_percent, gpu_util_percent, gpu_mem_util_percent, gpu_temp_c, gpu_core_clock_mhz, gpu_power_usage_w, gpu_power_limit_w, avg_read_latency_ms, avg_write_latency_ms, read_speed_bytes, write_speed_bytes, and disk_usage_percent.

The remaining 21 are rolling statistics computed across all 10 samples. For each of 7 key metrics (CPU percent, CPU frequency, RAM percent, swap percent, GPU clock, read speed, write speed), three statistics are calculated: rolling mean, rolling standard deviation, and linear slope using numpy.polyfit. These capture both the current level and the recent trend, which is needed for detecting gradual faults like memory leaks and thermal throttling that only become clear over a window of data.

Any missing values default to 0.0 to prevent NaN values propagating through the model. The feature column order is fixed at training time and stored inside model.pkl alongside the model so inference always uses the same alignment.

7.4 Label Generation

Rather than labelling training data by hand, a rule-based labelling approach was used. The LabelEngine.apply() function from labels.py is applied to every row in the training data, both synthetic and real, to generate boolean labels automatically from the same domain-knowledge thresholds used at inference time. This means that changing a threshold updates the labels for the entire dataset without any manual re-labelling. The label definitions were revised in Semester 2 Week 6 after identifying overlap in the earlier versions, which was straightforward to fix because of this approach.

7.5 Training Data

The training dataset is described in detail in Section 6.4. In total, over 20,000 rows were used across synthetic and real sources. Figure 7.1 shows the breakdown by source: 8,400 synthetic rows generated by generate_training_data.py, plus real telemetry collected from four separate .db files across three physical machines. The full dataset is saved to analytics_service/data/training_data.csv with a source column distinguishing real from synthetic rows. While collecting real telemetry, 3DMark [23] stress tests were run on the machines to push the hardware harder and try to trigger more fault conditions naturally, rather than relying entirely on normal workloads.

The synthetic data was generated by generate_training_data.py, which contains 12 scenario builders, one for each fault type. Each builder generates between 500 and 800 rows by defining the metric ranges that would be expected during that fault condition, for example high CPU percent and a dropping frequency slope for cpu_thermal_throttle, and then applying a random jitter of up to 12% on each metric to simulate natural variation. The metric ranges were defined manually based on hardware knowledge and research into typical performance limits. This approach meant every label boundary condition and edge case could be covered in a controlled way, which would not be possible to guarantee from real telemetry alone.

Figure 7.2 shows the label frequency across the dataset, split by source. Most labels sit between 600 and 1,200 rows, which is enough for the model to learn each fault type. disk_full is the most common label with over 3,000 rows, the majority of which came from real telemetry, since disk usage above 90% was a genuine condition recorded on the monitoring machines. GPU labels such as gpu_overheating, gpu_power_throttle, and gpu_vram_pressure are almost entirely synthetic, because those fault conditions were not triggered on the real machines during data collection. This is expected given that the real sessions were recorded under normal workloads.

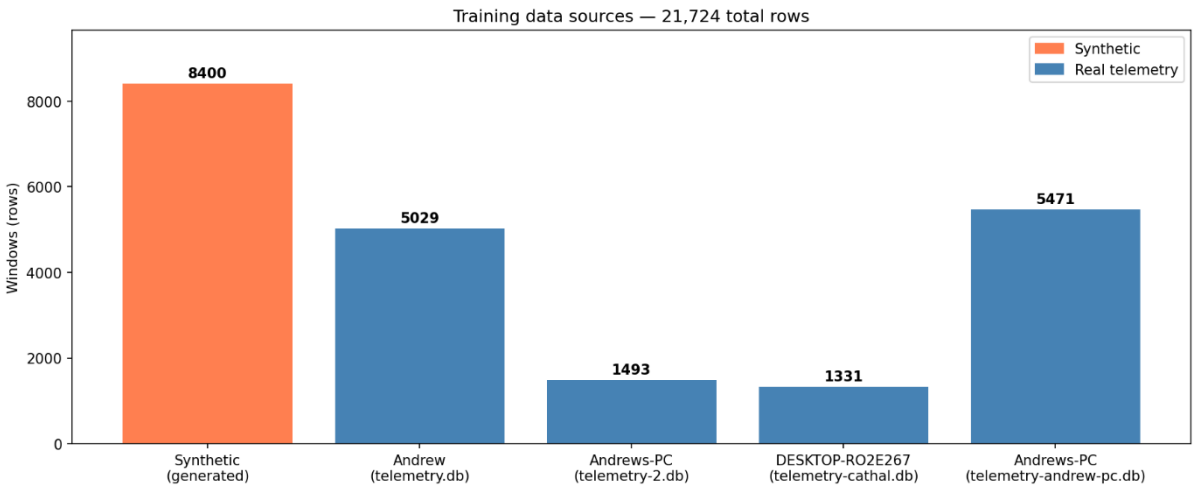


Figure 7.1 - Training data sources showing the breakdown of over 20,000 rows across synthetic generated data and real telemetry collected from three physical machines.

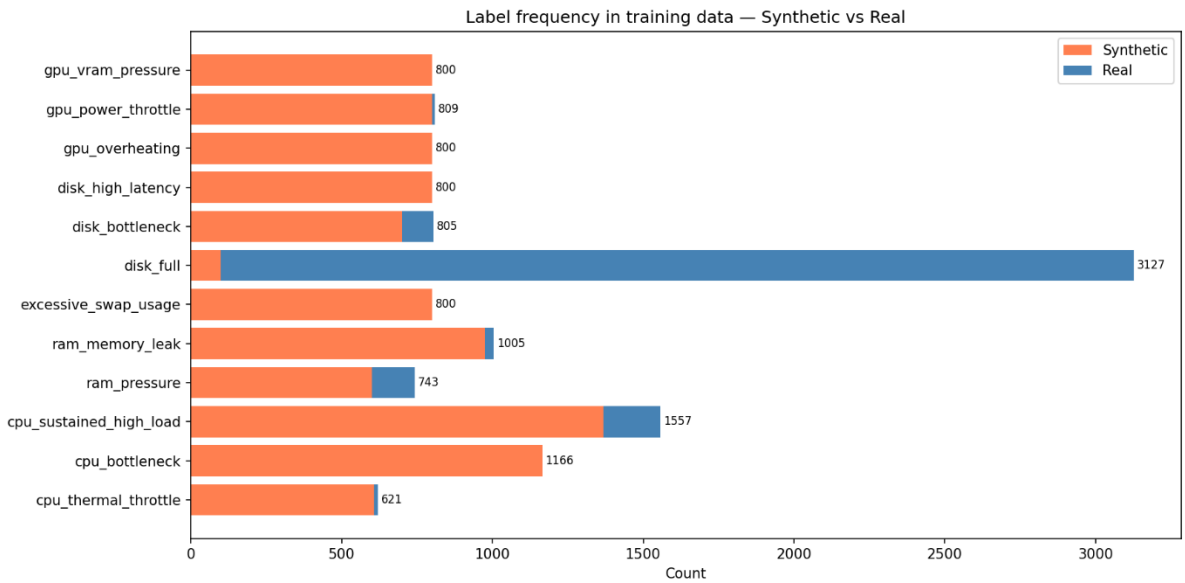


Figure 7.2 - Label frequency across all 12 fault labels in the training dataset, split by synthetic and real source.

7.6 Model Training and Results

The model was trained using MultiOutputClassifier wrapping a RandomForestClassifier with 200 estimators, min_samples_leaf=2, and class_weight='balanced'. The balanced class weight compensates for the natural imbalance between healthy samples and fault samples in the dataset. An 80/20 train-test split was used with random_state=42 for reproducibility. Training is done once offline via train.py and is not repeated at runtime.

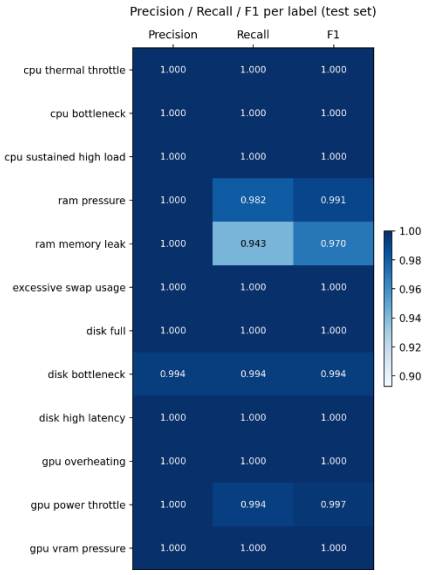
Figure 7.3 shows the per-label precision, recall, and F1 scores on the test set. Ten of the twelve labels achieved a perfect F1 of 1.000. The two lowest scoring labels were ram_memory_leak at 0.970 and ram_pressure at 0.991. Both are RAM labels where the trigger condition depends on rolling slopes across the 10-sample window, which

are more gradual and harder to separate cleanly than point-in-time thresholds. The confusion matrices in Figure 7.4 confirm this: ram_memory_leak had 12 false negatives and ram_pressure had 3, while every other label had zero or one missed prediction across the entire test set.

Figures 7.5 and 7.6 show two sample decision trees extracted from individual trees inside the Random Forest. The gpu_power_throttle tree illustrates how closely the model learned the label rule: the root split checks whether gpu_power_usage_w is below 149W, and if it is not, the second split checks whether the GPU clock rolling slope is below -5, which is almost exactly the condition defined in labels.py. The cpu_sustained_high_load tree is more interesting. The root split is on gpu_power_usage_w, not CPU percent, which initially looks unexpected. This is because GPU power usage is a strong proxy for whether the machine is under a GPU-heavy workload, and under those conditions CPU sustained load behaves differently. The second level then splits on cpu_percent_total_roll_mean at around 88, matching the label threshold directly.

Figure 7.7 shows the label co-occurrence matrix, which records what percentage of a label's positive rows also had another label fire at the same time. The strongest relationship is between cpu_bottleneck and cpu_sustained_high_load, which co-occur 84% of the time, since a CPU that has been sustaining high load will often cross into bottleneck territory. cpu_thermal_throttle and cpu_bottleneck co-occur 24% of the time, which makes sense as a CPU being throttled is often also the bottleneck. Most other labels are largely independent of each other, particularly across component boundaries, which confirms that the label definitions are well separated and not just detecting the same underlying condition with different names.

The trained model is serialised to analytics_service/data/model.pkl as a dictionary containing the model object, the feature column names, and the label names. It is loaded once at startup by PerformanceModel with no retraining at runtime.



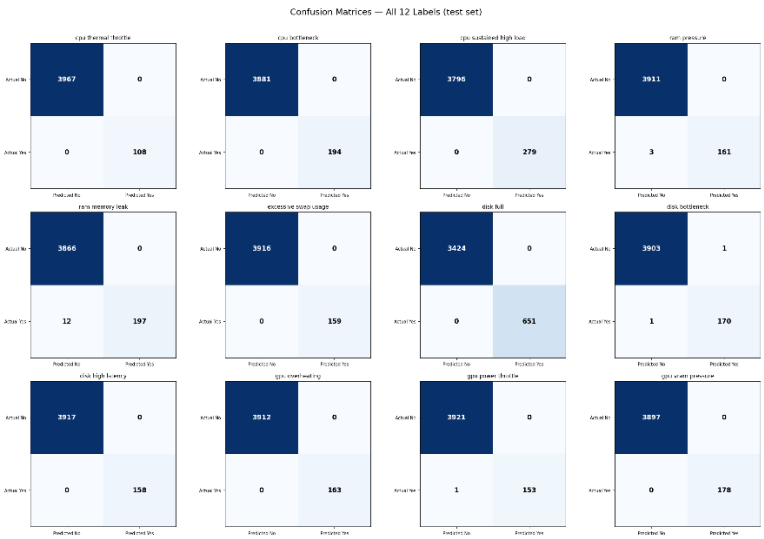


Figure 7.3 (Left) - Per-label precision, recall, and F1 scores on the 20% test set.
Figure 7.4 (Right) - Confusion matrices for all 12 labels on the test set.

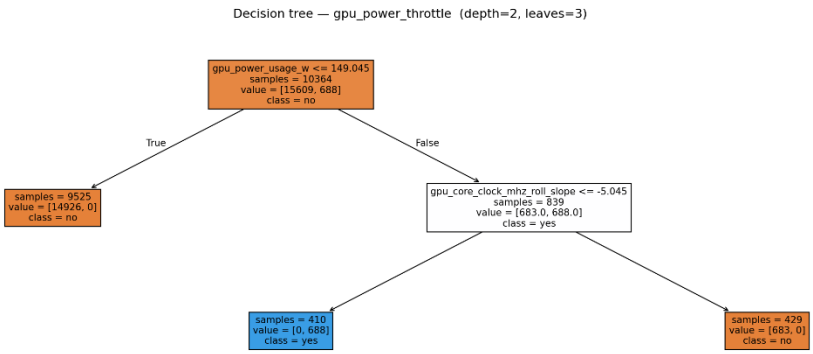


Figure 7.5 - A single decision tree from the Random Forest for gpu_power_throttle

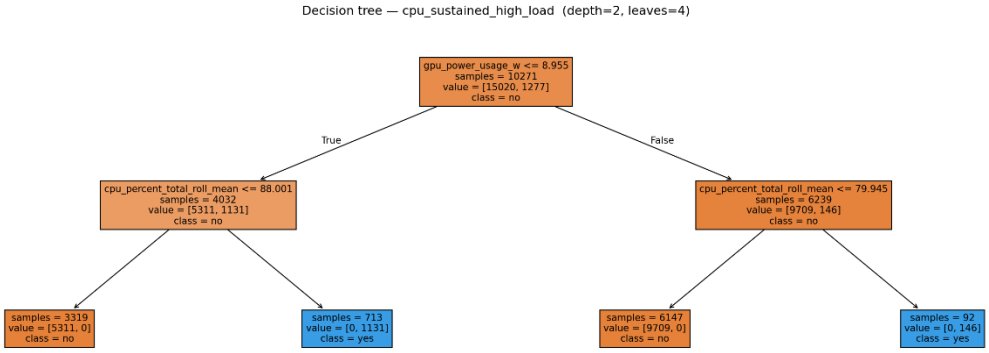


Figure 7.6 - A single decision tree from the Random Forest for cpu_sustained_high_load.

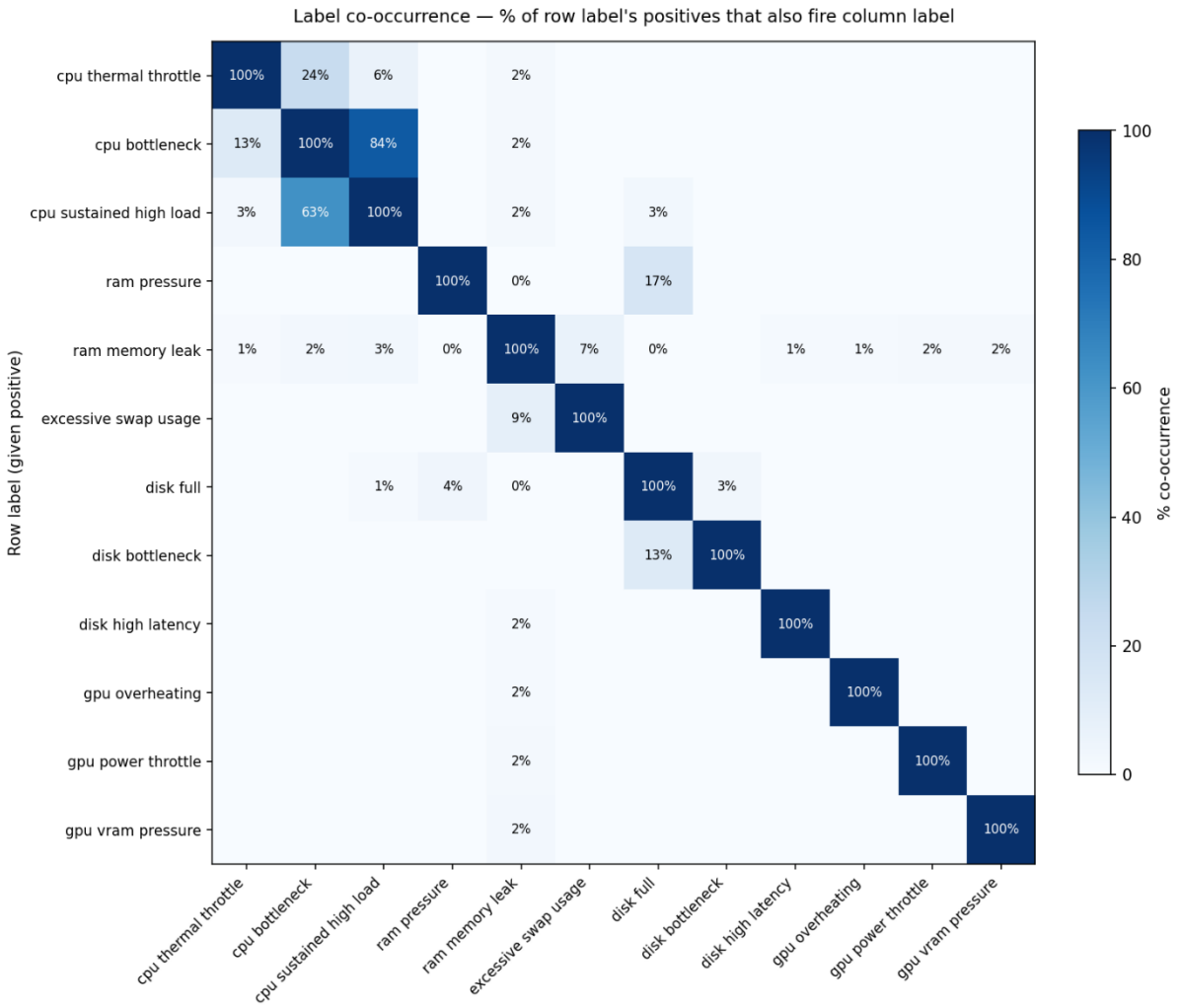


Figure 7.7 - Label co-occurrence matrix showing what percentage of a label's positive rows also had another label fire.

7.7 Inference and Dashboard Integration

The Analytics tab is not started automatically. When the user clicks the Start Analytics button, AnalyticsThread launches as a background QThread. It opens its own read-only connection to telemetry.db and polls the sample count once per second during the warmup period. The Model Status card shows "Collecting..." with a progress count until 100 samples have been gathered. Once that threshold is reached, the status switches to "Ready" and inference runs every 5 seconds.

Each cycle reads the last 10 samples from the database, builds the feature vector using FeatureExtractor.compute(), and passes it to PerformanceModel.predict(). The model calls predict() for the binary label output and predict_proba() for per-label confidence probabilities. predict_proba() returns a list of arrays, one per label, where index 1 is the probability of the label firing. The method also calculates the health score and component risk levels, then emits everything back to the UI thread via a Qt signal.

The Analytics panel has three sections. At the top is a row of stat cards showing the health score on the left, a session statistics bar in the centre displaying running time, predictions made, and issues flagged, and the model status card on the right. Below

that is the Detected Issues panel, a scrollable list of all 12 issue cards sorted by confidence from highest to lowest. Each card shows a component badge, the issue title, and the model's confidence percentage for that label. If an issue has fired, the card is highlighted and shows a plain-language description of the problem and a suggested fix below the title. Unfired issues are shown in a muted style so the user can see what the model is monitoring even when everything is healthy.

New alerts are only added to the alert log the first time an issue fires, not on every prediction cycle. This prevents a sustained fault from flooding the log with duplicate entries. The Alerts button in the top bar opens a dialog showing every unique alert recorded during the session with its timestamp, component, title, and confidence.

The Export Analytics Data button opens a dialog that lets the user choose what to include in the export. The two options are a session summary, covering running time, total predictions made, issues flagged, and the current health score, and a detected issues export showing each fired label with its component, confidence percentage, and plain-language description. The output is saved as a CSV file with a timestamped filename. This is separate from the Export DB button in the top bar, which exports the full raw telemetry database.

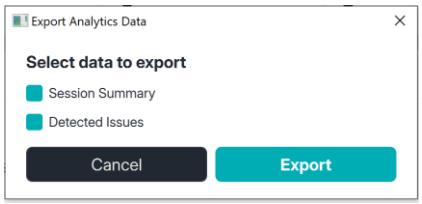


Figure: Export Analytics Data popup

7.8 Health Score Formula

The health score is a single integer between 0 and 100 displayed on the Analytics panel. It is calculated using a severity-weighted, confidence-scaled penalty formula defined in model.py.

Each of the 12 labels has a severity rating from 1 to 5 defined in LABEL_SEVERITY in labels.py. Each severity level maps to a penalty value: 1 gives 2 points, 2 gives 5 points, 3 gives 15 points, 4 gives 35 points, and 5 gives 60 points. The steep increase ensures minor issues barely affect the score while critical issues dominate it. When a label fires, its penalty is multiplied by the model's confidence probability for that label, so a label detected at 90% confidence penalises the score more than the same label at 55%. All per-label penalties are summed and subtracted from 100, then clamped and rounded: health_score = 100 – Severity x Confidence.

Current severity assignments from labels.py:

Severity	Labels
5 - Critical	gpu_overheating
4 - Serious	cpu_thermal_throttle, ram_memory_leak, disk_full
3 - Warning	cpu_bottleneck, ram_pressure, excessive_swap_usage, disk_high_latency, gpu_power_throttle, gpu_vram_pressure

Severity	Labels
2 - Minor	cpu_sustained_high_load, disk_bottleneck

Multiple simultaneous issues accumulate penalties, so the score responds to both the severity and the certainty of what the model detects. A machine with only a minor disk bottleneck at 60% confidence would score around 97. A machine with GPU overheating at 90% confidence would score around 46.

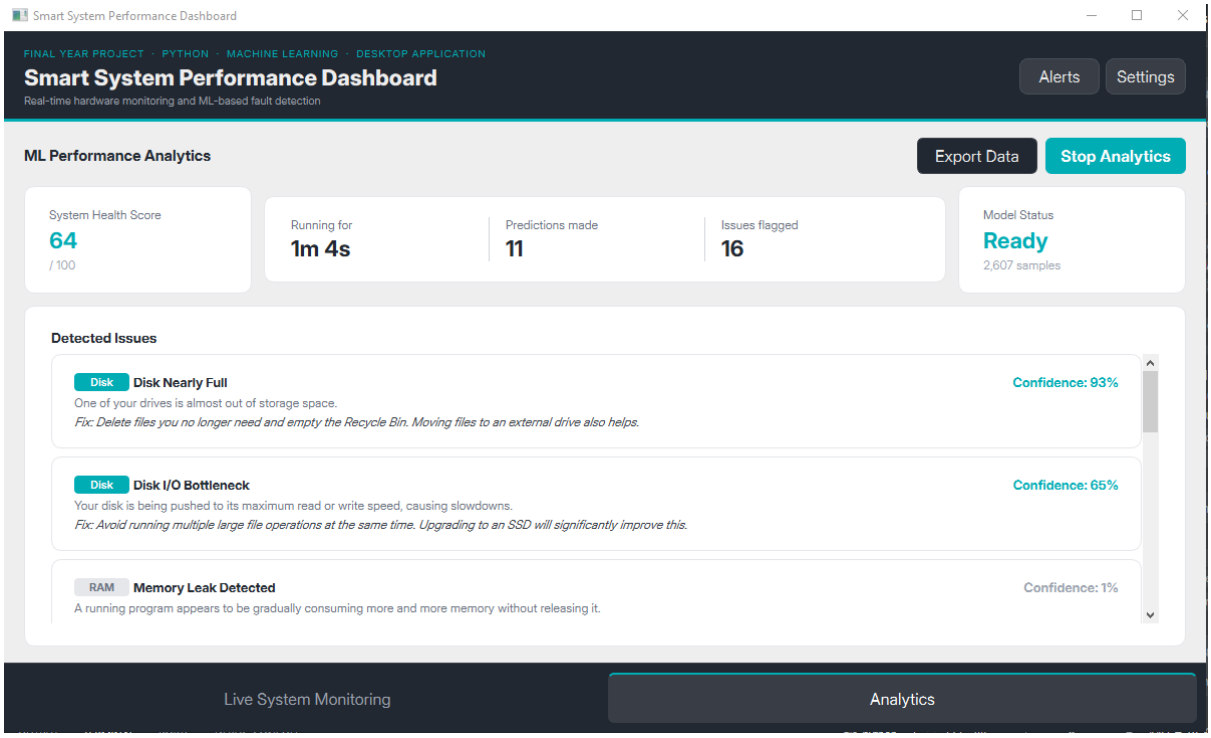


Figure: Analytics Panel screenshot showing health score, detected issues, and component risk indicators

8. Testing

8.1 Test Strategy

Testing is organised into three suites using pytest, one for each independently testable service: the Collector, Storage, and Analytics services. All three suites pass. The total test code spans approximately 880 lines across four files: test_collectors.py (310 lines), test_storage.py (168 lines), test_labels.py (179 lines), and test_model.py (224 lines), covering over 80 individual tests in total.

The tests cover three levels. At the unit level, each collector and each label rule is tested in isolation, the collector tests run against the live Windows hardware APIs and verify that each class returns the correct keys, value ranges, and handles edge cases without crashing. The label tests verify each of the 12 rules individually against controlled feature dictionaries. At the integration level, test_model.py tests the full analytics pipeline end-to-end, building a raw sample window, passing it through FeatureExtractor, and running it through PerformanceModel, verifying that the combined pipeline produces the correct output structure and fires the correct labels. At the system level, the application was deployed and run on three physical machines covering different operating systems and hardware configurations, with the results inspected directly, as described in section 8.6.

The Storage tests use an in-memory SQLite database via db_path=':memory:' so they are fully isolated with no file system dependency. The Analytics tests verify both the rule-based label engine and the end-to-end model inference pipeline using controlled synthetic feature vectors.

8.2 Collector Tests (test_collectors.py, 310 lines)

The collector tests cover all four hardware collectors. Each is tested for the correct return key structure, value ranges, and logical constraints.

For CPU, the tests check that cpu_percent_total is within 0-100, freq_current_mhz is positive when not None, and the timestamp is a string. An edge case test verifies that when the PDH query object is unavailable, the collector falls back gracefully without raising an exception.

For RAM, the tests verify that used_ram_gb never exceeds total_ram_gb, both usage percentages are within 0-100, and total RAM is positive. Edge cases cover zero swap and a simulated high RAM usage scenario at 97.5%.

For Disk, the tests confirm read and write speeds are non-negative, partition usage is within 0-100, and used never exceeds total. Edge cases include a PermissionError on one partition being skipped while the other continues, a disk at 99.6% usage, and a completely empty disk. A multiple-partition test mocks two drives and confirms both are returned correctly.

For GPU, a no-GPU test mocks zero devices and confirms the collector returns an empty gpus list and gpu_count of 0 without raising an exception. The gpu_available flag is verified to be a boolean. When a GPU is present, utilisation, VRAM, temperature, and power values are all checked for valid ranges and the power usage is confirmed to not exceed the power limit by more than 5%.

```
============================================================================ test session starts ============================================================================
platform win32 -- Python 3.12.6, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\zee88\Final-Year-Project-1\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\zee88\Final-Year-Project-1
collected 36 items

collector_service/tests/test_collectors.py::TestCPUCollector::test_returns_required_keys PASSED                                                                          [  2%]
collector_service/tests/test_collectors.py::TestCPUCollector::test_cpu_percent_in_range PASSED                                                                           [  5%]
collector_service/tests/test_collectors.py::TestCPUCollector::test_freq_current_is_positive PASSED                                                                       [  8%]
collector_service/tests/test_collectors.py::TestCPUCollector::test_timestamp_is_string PASSED                                                                            [ 11%]
collector_service/tests/test_collectors.py::TestRAMCollector::test_returns_required_keys PASSED                                                                          [ 13%]
collector_service/tests/test_collectors.py::TestRAMCollector::test_ram_usage_in_range PASSED                                                                             [ 16%]
collector_service/tests/test_collectors.py::TestRAMCollector::test_swap_usage_in_range PASSED                                                                            [ 19%]
collector_service/tests/test_collectors.py::TestRAMCollector::test_used_does_not_exceed_total PASSED                                                                      [ 22%]
collector_service/tests/test_collectors.py::TestRAMCollector::test_total_ram_is_positive PASSED                                                                           [ 25%]
collector_service/tests/test_collectors.py::TestRAMCollector::test_timestamp_is_string PASSED                                                                            [ 27%]
collector_service/tests/test_collectors.py::TestDiskCollector::test_returns_required_keys PASSED                                                                          [ 30%]
collector_service/tests/test_collectors.py::TestDiskCollector::test_disks_list_not_empty PASSED                                                                           [ 33%]
collector_service/tests/test_collectors.py::TestDiskCollector::test_read_speed_not_negative PASSED                                                                        [ 36%]
collector_service/tests/test_collectors.py::TestDiskCollector::test_write_speed_not_negative PASSED                                                                       [ 38%]
collector_service/tests/test_collectors.py::TestDiskCollector::test_disk_usage_in_range PASSED                                                                            [ 41%]
collector_service/tests/test_collectors.py::TestDiskCollector::test_disk_used_does_not_exceed_total PASSED                                                                [ 44%]
collector_service/tests/test_collectors.py::TestDiskCollector::test_disk_has_required_keys PASSED                                                                         [ 47%]
collector_service/tests/test_collectors.py::TestDiskCollector::test_multiple_partitions PASSED                                                                            [ 50%]
collector_service/tests/test_collectors.py::TestCPUCollectorEdgeCases::test_pdh_unavailable_falls_back_to_psutil PASSED                                                  [ 52%]
collector_service/tests/test_collectors.py::TestCPUCollectorEdgeCases::test_cpu_percent_not_negative PASSED                                                               [ 55%]
collector_service/tests/test_collectors.py::TestRAMCollectorEdgeCases::test_zero_swap PASSED                                                                             [ 58%]
collector_service/tests/test_collectors.py::TestRAMCollectorEdgeCases::test_high_ram_usage PASSED                                                                        [ 61%]
collector_service/tests/test_collectors.py::TestDiskCollectorEdgeCases::test_permission_error_partition_skipped PASSED                                                    [ 63%]
collector_service/tests/test_collectors.py::TestDiskCollectorEdgeCases::test_disk_nearly_full PASSED                                                                      [ 66%]
collector_service/tests/test_collectors.py::TestDiskCollectorEdgeCases::test_disk_empty PASSED                                                                            [ 69%]
collector_service/tests/test_collectors.py::TestGPUCollectorNoGPU::test_no_gpu_returns_empty_list PASSED                                                                  [ 72%]
collector_service/tests/test_collectors.py::TestGPUCollectorNoGPU::test_no_gpu_frontend_guard PASSED                                                                      [ 75%]
collector_service/tests/test_collectors.py::TestGPUCollectorNoGPU::test_gpu_available_false_when_nvml_unavailable PASSED                                                  [ 77%]
collector_service/tests/test_collectors.py::TestGPUCollectorNoGPU::test_gpu_available_flag_is_bool PASSED                                                                 [ 80%]
collector_service/tests/test_collectors.py::TestGPUCollector::test_returns_required_keys PASSED                                                                           [ 83%]
collector_service/tests/test_collectors.py::TestGPUCollector::test_gpu_count_matches_list PASSED                                                                          [ 86%]
collector_service/tests/test_collectors.py::TestGPUCollector::test_gpu_util_in_range PASSED                                                                               [ 88%]
collector_service/tests/test_collectors.py::TestGPUCollector::test_gpu_mem_util_in_range PASSED                                                                           [ 91%]
collector_service/tests/test_collectors.py::TestGPUCollector::test_gpu_temp_is_positive PASSED                                                                            [ 94%]
collector_service/tests/test_collectors.py::TestGPUCollector::test_gpu_power_usage_does_not_exceed_limit PASSED                                                           [ 97%]
collector_service/tests/test_collectors.py::TestGPUCollector::test_gpu_has_required_keys PASSED                                                                           [100%]

============================================================================= 36 passed in 0.88s ==============================================================================
```

Figure: pytest output showing all collector tests passing

8.3 Storage Tests (test_storage.py, 168 lines)

All storage tests use a fresh in-memory StorageManager fixture, so each test starts with a clean database.

Host registration tests verify that host_uuid is set, is a string, appears in the host table, and that the MAC address is stored correctly. Session tests confirm that session_id is a positive integer, exists in the session table, and is linked to the correct host_uuid.

Sample insertion is tested by checking the count increments correctly after each insert_sample() call and that five consecutive inserts produce a count of five. Read tests confirm that get_recent_samples() returns an empty list before any inserts, returns the correct number of rows after inserts, respects the n limit parameter, and returns dictionaries. Data integrity is verified by checking that CPU percent, RAM percent, and disk usage values written to the database match what is read back via get_recent_samples().

```
$ python -m pytest storage_service/tests/test_storage.py -v
============================================================================ test session starts ============================================================================
platform win32 -- Python 3.12.6, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\zee88\Final-Year-Project-1\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\zee88\Final-Year-Project-1
collected 19 items

storage_service/tests/test_storage.py::TestHostRegistration::test_host_uuid_is_set PASSED                                                                                [  5%]
storage_service/tests/test_storage.py::TestHostRegistration::test_host_uuid_is_string PASSED                                                                              [ 10%]
storage_service/tests/test_storage.py::TestHostRegistration::test_host_registered_in_db PASSED                                                                            [ 15%]
storage_service/tests/test_storage.py::TestHostRegistration::test_host_has_cpu_max_mhz PASSED                                                                             [ 21%]
storage_service/tests/test_storage.py::TestHostRegistration::test_host_mac_address_stored PASSED                                                                           [ 26%]
storage_service/tests/test_storage.py::TestSession::test_session_id_is_set PASSED                                                                                          [ 31%]
storage_service/tests/test_storage.py::TestSession::test_session_registered_in_db PASSED                                                                                   [ 36%]
storage_service/tests/test_storage.py::TestSession::test_session_linked_to_host PASSED                                                                                     [ 42%]
storage_service/tests/test_storage.py::TestInsertSample::test_insert_sample_no_error PASSED                                                                                [ 47%]
storage_service/tests/test_storage.py::TestInsertSample::test_sample_count_increments PASSED                                                                               [ 52%]
storage_service/tests/test_storage.py::TestInsertSample::test_insert_multiple_samples PASSED                                                                               [ 57%]
storage_service/tests/test_storage.py::TestReadSamples::test_get_recent_samples_empty PASSED                                                                               [ 63%]
storage_service/tests/test_storage.py::TestReadSamples::test_get_recent_samples_returns_inserted PASSED                                                                    [ 68%]
storage_service/tests/test_storage.py::TestReadSamples::test_get_recent_samples_respects_limit PASSED                                                                      [ 73%]
storage_service/tests/test_storage.py::TestReadSamples::test_get_recent_samples_returns_dicts PASSED                                                                       [ 78%]
storage_service/tests/test_storage.py::TestReadSamples::test_get_recent_samples_all_sessions_returns_inserted PASSED                                                       [ 84%]
storage_service/tests/test_storage.py::TestReadSamples::test_sample_contains_cpu_data PASSED                                                                               [ 89%]
storage_service/tests/test_storage.py::TestReadSamples::test_sample_contains_ram_data PASSED                                                                               [ 94%]
storage_service/tests/test_storage.py::TestReadSamples::test_sample_contains_disk_data PASSED                                                                              [100%]

============================================================================= 19 passed in 0.63s ==============================================================================
```

Figure: pytest output showing all Storage tests passing

8.4 Analytics Tests (test_labels.py, 179 lines; test_model.py, 224 lines)

test_labels.py starts with a healthy baseline fixture with all metrics in normal ranges and confirms that no labels fire. Each of the 12 labels is then tested individually. For each label, there is a test that confirms it fires when its threshold conditions are met, and one or more tests that confirm it does not fire when conditions are just outside the threshold. Combined conditions are also checked, for example ram_pressure requires both high RAM and low CPU, so the test verifies it does not fire when CPU is also high.

test_model.py verifies the full end-to-end pipeline from raw sample window through to model output. The model loading tests confirm that _feature_cols is populated, the correct 12 label names are present, and the model object is not None. Output structure tests confirm that predict() returns a dictionary containing issues, component_risks, and probabilities keys, that all four components (CPU, RAM, Disk, GPU) are present in component_risks, and that component risk values are 0, 1, or 2. Nine fault scenario windows are tested, each verifying the correct label fires: cpu_sustained_high_load, cpu_bottleneck, cpu_thermal_throttle, ram_pressure, excessive_swap_usage, disk_full, disk_high_latency, gpu_overheating, and gpu_vram_pressure. A robustness test confirms that a missing feature key defaults to 0.0 without crashing inference.

```
$ python -m pytest analytics_service/tests/test_labels.py -v
========================================================================== test session starts ==========================================================================
platform win32 -- Python 3.12.6, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\zee88\Final-Year-Project-1\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\zee88\Final-Year-Project-1
collected 27 items

analytics_service/tests/test_labels.py::TestHealthyBaseline::test_no_labels_fire_on_healthy_system PASSED                                                    [  3%]
analytics_service/tests/test_labels.py::TestCPULabels::test_cpu_thermal_throttle_fires PASSED                                                                [  7%]
analytics_service/tests/test_labels.py::TestCPULabels::test_cpu_thermal_throttle_no_fire_low_cpu PASSED                                                      [ 11%]
analytics_service/tests/test_labels.py::TestCPULabels::test_cpu_thermal_throttle_no_fire_stable_freq PASSED                                                  [ 14%]
analytics_service/tests/test_labels.py::TestCPULabels::test_cpu_bottleneck_fires PASSED                                                                      [ 18%]
analytics_service/tests/test_labels.py::TestCPULabels::test_cpu_bottleneck_no_fire_high_ram PASSED                                                           [ 22%]
analytics_service/tests/test_labels.py::TestCPULabels::test_cpu_sustained_high_load_fires PASSED                                                             [ 25%]
analytics_service/tests/test_labels.py::TestCPULabels::test_cpu_sustained_high_load_no_fire PASSED                                                           [ 29%]
analytics_service/tests/test_labels.py::TestRAMLabels::test_ram_pressure_fires PASSED                                                                        [ 33%]
analytics_service/tests/test_labels.py::TestRAMLabels::test_ram_pressure_no_fire_high_cpu PASSED                                                             [ 37%]
analytics_service/tests/test_labels.py::TestRAMLabels::test_ram_memory_leak_fires PASSED                                                                     [ 40%]
analytics_service/tests/test_labels.py::TestRAMLabels::test_ram_memory_leak_no_fire_swap_stable PASSED                                                       [ 44%]
analytics_service/tests/test_labels.py::TestRAMLabels::test_excessive_swap_fires PASSED                                                                      [ 48%]
analytics_service/tests/test_labels.py::TestRAMLabels::test_excessive_swap_no_fire PASSED                                                                    [ 51%]
analytics_service/tests/test_labels.py::TestDiskLabels::test_disk_full_fires PASSED                                                                           [ 55%]
analytics_service/tests/test_labels.py::TestDiskLabels::test_disk_full_no_fire PASSED                                                                         [ 59%]
analytics_service/tests/test_labels.py::TestDiskLabels::test_disk_bottleneck_fires PASSED                                                                     [ 62%]
analytics_service/tests/test_labels.py::TestDiskLabels::test_disk_bottleneck_no_fire_high_cpu PASSED                                                          [ 66%]
analytics_service/tests/test_labels.py::TestDiskLabels::test_disk_high_latency_fires_read PASSED                                                              [ 70%]
analytics_service/tests/test_labels.py::TestDiskLabels::test_disk_high_latency_fires_write PASSED                                                             [ 74%]
analytics_service/tests/test_labels.py::TestDiskLabels::test_disk_high_latency_no_fire PASSED                                                                 [ 77%]
analytics_service/tests/test_labels.py::TestGPULabels::test_gpu_overheating_fires PASSED                                                                      [ 81%]
analytics_service/tests/test_labels.py::TestGPULabels::test_gpu_overheating_no_fire PASSED                                                                    [ 85%]
analytics_service/tests/test_labels.py::TestGPULabels::test_gpu_power_throttle_fires PASSED                                                                   [ 88%]
analytics_service/tests/test_labels.py::TestGPULabels::test_gpu_power_throttle_no_fire_clock_stable PASSED                                                    [ 92%]
analytics_service/tests/test_labels.py::TestGPULabels::test_gpu_vram_pressure_fires PASSED                                                                    [ 96%]
analytics_service/tests/test_labels.py::TestGPULabels::test_gpu_vram_pressure_no_fire PASSED                                                                  [100%]

========================================================================== 27 passed in 0.06s ==========================================================================
$ python -m pytest analytics_service/tests/test_model.py -v
========================================================================== test session starts ==========================================================================
platform win32 -- Python 3.12.6, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\zee88\Final-Year-Project-1\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\zee88\Final-Year-Project-1
collected 26 items

analytics_service/tests/test_model.py::TestModelLoads::test_model_has_feature_cols PASSED                                                                    [  3%]
analytics_service/tests/test_model.py::TestModelLoads::test_model_has_correct_label_names PASSED                                                              [  7%]
analytics_service/tests/test_model.py::TestModelLoads::test_model_object_exists PASSED                                                                        [ 11%]
analytics_service/tests/test_model.py::TestPredictOutputStructure::test_predict_returns_dict PASSED                                                           [ 15%]
analytics_service/tests/test_model.py::TestPredictOutputStructure::test_predict_has_issues_key PASSED                                                         [ 19%]
analytics_service/tests/test_model.py::TestPredictOutputStructure::test_predict_has_component_risks_key PASSED                                                [ 23%]
analytics_service/tests/test_model.py::TestPredictOutputStructure::test_issues_is_list PASSED                                                                 [ 26%]
analytics_service/tests/test_model.py::TestPredictOutputStructure::test_component_risks_has_all_components PASSED                                             [ 30%]
analytics_service/tests/test_model.py::TestPredictOutputStructure::test_component_risk_values_are_0_1_or_2 PASSED                                             [ 34%]
analytics_service/tests/test_model.py::TestPredictOutputStructure::test_issues_only_contains_valid_labels PASSED                                               [ 38%]
analytics_service/tests/test_model.py::TestHealthyPrediction::test_healthy_window_no_issues PASSED                                                            [ 42%]
analytics_service/tests/test_model.py::TestHealthyPrediction::test_healthy_window_all_risks_zero PASSED                                                        [ 46%]
analytics_service/tests/test_model.py::TestIssueDetection::test_detects_cpu_sustained_high_load PASSED                                                         [ 50%]
analytics_service/tests/test_model.py::TestIssueDetection::test_detects_cpu_bottleneck PASSED                                                                 [ 53%]
analytics_service/tests/test_model.py::TestIssueDetection::test_detects_cpu_thermal_throttle PASSED                                                           [ 57%]
analytics_service/tests/test_model.py::TestIssueDetection::test_detects_ram_pressure PASSED                                                                   [ 61%]
analytics_service/tests/test_model.py::TestIssueDetection::test_detects_excessive_swap PASSED                                                                 [ 65%]
analytics_service/tests/test_model.py::TestIssueDetection::test_detects_disk_full PASSED                                                                      [ 69%]
analytics_service/tests/test_model.py::TestIssueDetection::test_detects_disk_high_latency PASSED                                                              [ 73%]
analytics_service/tests/test_model.py::TestIssueDetection::test_detects_gpu_overheating PASSED                                                                [ 76%]
analytics_service/tests/test_model.py::TestIssueDetection::test_detects_gpu_vram_pressure PASSED                                                              [ 80%]
analytics_service/tests/test_model.py::TestComponentRisks::test_cpu_risk_1_for_one_issue PASSED                                                               [ 84%]
analytics_service/tests/test_model.py::TestComponentRisks::test_gpu_risk_0_on_healthy PASSED                                                                  [ 88%]
analytics_service/tests/test_model.py::TestComponentRisks::test_disk_risk_1_when_full PASSED                                                                  [ 92%]
analytics_service/tests/test_model.py::TestComponentRisks::test_ram_risk_1_when_pressured PASSED                                                              [ 96%]
analytics_service/tests/test_model.py::TestComponentRisks::test_missing_feature_does_not_crash PASSED                                                         [100%]

========================================================================== 26 passed in 17.96s ==========================================================================
```

Figure: pytest output showing analytics – model and label tests passing

8.5 Performance Testing

Collector performance was profiled using debug_timing.py, timing each collector individually and then all four together over 10 iterations, before and after the GPU optimisation. The full profiling output before and after is shown in Section 5.2.

Collector	Before Optimisation (avg)	After Optimisation (avg)	Change
CPU	~1.94 ms	~0.21ms	-89%
RAM	~67.73 ms	~7.48ms	+380%
GPU	~344.58 ms	~17.57ms	-95%
Disk	~1.22ms	~5.51ms	+351%
All Together	~468ms avg, spikes to 1,208ms	~28ms avg, max 38ms	~95% reduction

The GPU result is the headline improvement. Before the fix it was ranging from 112ms to over 1,200ms per iteration, with the variance caused by the pynvml session being initialised and shut down on every call, adding unpredictable overhead depending on driver state. After moving initialisation to module level and removing the PCIe metrics, GPU settled consistently between 8ms and 25ms.

The CPU reading improved significantly as well, dropping from around 1.94ms to under 0.25ms. This is likely because the PDH query was also being set up more efficiently as part of the same optimisation pass, reducing the per-call overhead.

The RAM and Disk figures going up between the two profiling runs is not a regression. Between the two test runs, additional metrics were added to both collectors as the project progressed. The RAM collector was extended to include swap usage alongside the core RAM metrics, and the Disk collector was extended to calculate read and write latency in addition to speeds, which requires an extra set of I/O counter comparisons per call. The increase in time is proportional to the extra work being done, and both collectors still sit well under 10ms per call, which is well within the acceptable range.

The overall collection cycle time dropping from an average of 468ms with spikes to 1,208ms, down to a consistent 22 to 38ms, is the result that matters. This meets the non-functional requirement that each collection cycle must complete within 50ms to prevent GUI freezing, and means the GUI thread is never blocked waiting for collection to finish.

8.6 Multi-Machine Testing

The application was tested on three physical machines to verify cross-platform and cross-hardware compatibility. The first was my laptop running Windows 11 with an NVIDIA GPU. The second was my desktop PC running Windows 10 with an NVIDIA GPU. The third was a school computer running Windows 11 with no discrete GPU.

Two issues were discovered and fixed during this testing phase. On my desktop PC running Windows 10, the CPU frequency reading was returning an incorrect fixed value because the Windows 11 PDH Actual Frequency counter is not available on

Windows 10. This led to the Windows 10 fallback being implemented, using the % Processor Performance counter multiplied by the base clock frequency. On the school computer with no GPU, pynvml was throwing errors at startup when no NVIDIA hardware was detected. This was fixed by wrapping the initialisation in a try-except and setting gpu_available = False, which causes all GPU-dependent code paths to be skipped cleanly for the rest of the session.

On each machine the application was launched, allowed to collect several minutes of telemetry, and the resulting telemetry.db file was inspected to confirm correct host UUID generation, session creation, and sample insertion. My laptop and desktop both produced valid database files that were later used as part of the real telemetry training data described in Section 6.4. The school computer was used for compatibility testing only.

9. Problem Solving and Challenges

9.1 GPU Collector Bottleneck and GUI Freeze

The most significant issue during development was a 0.5 to 1.0 second GUI freeze on every data collection cycle. A structured five-step root cause analysis was carried out, documented in the weekly log for Semester 2 Weeks 4 and 5 and described in full in Section 5.2. In short, profiling identified the GPU collector as the cause, averaging 344ms per call compared to under 8ms for the other three collectors. Two issues were found inside gpu_collector.py: the PCIe throughput metrics were significantly slower than all other NVML queries, and the pynvml session was being initialised and shut down on every single collection call. Removing the PCIe metrics and moving initialisation to module level brought the GPU collector down to an average of 22ms, with full collection cycles dropping from an average of 468ms to a consistent 22 to 38ms.

9.2 CPU Frequency Reporting Inaccuracy

psutil.cpu_freq() was returning a fixed value of 2304 MHz on my laptop regardless of load. Investigation revealed that Windows reports the nominal base clock to psutil rather than the real-time frequency. The fix was to use the Windows PDH counter Processor Information(_Total)\Actual Frequency via pywin32, which reads from the hardware and accounts for turbo boost. This returned accurate real-time values, for example 904 MHz at idle and 3216 MHz under load on a machine with a 2304 MHz base clock. A Windows 10 fallback was also implemented using % Processor Performance multiplied by the base frequency from the Windows Registry, since the Actual Frequency counter is not available on Windows 10.

9.3 Database Session Overlap Bug

A bug was found where disk partition data from a previous session appeared in the current session's records. The root cause was that StorageManager was passing the wrong variable as partition_id when inserting disk_partition_sample rows, causing each session's disk data to reference the previous session's partition registry entries. The fix was a one-line variable correction.

```
SELECT * FROM `sample` LIMIT 30,30
```

Execute

sample_id	session_id	ts_iso	ts_unix_ms	collect_duration_ms	dropped_metrics
31	2	2026-03-08T15:09:31.498377	1772982571498	0	1
32	2	2026-03-08T15:09:32.530743	1772982572530	0	1
33	2	2026-03-08T15:09:33.562554	1772982573562	0	1
34	2	2026-03-08T15:09:34.594792	1772982574594	0	1
35	2	2026-03-08T15:09:35.621255	1772982575621	0	1
36	2	2026-03-08T15:09:36.652664	1772982576652	0	1
37	2	2026-03-08T15:09:37.679406	1772982577679	0	1
38	2	2026-03-08T15:09:38.714376	1772982578714	0	1
39	2	2026-03-08T15:09:39.741864	1772982579741	0	1
40	2	2026-03-08T15:09:40.800103	1772982580800	0	1
41	2	2026-03-08T15:09:41.838220	1772982581838	0	1
42	2	2026-03-08T15:09:42.872718	1772982582872	0	1
43	2	2026-03-08T15:09:43.908231	1772982583908	0	1
44	2	2026-03-08T15:09:44.943625	1772982584943	0	1
45	2	2026-03-08T15:09:45.974737	1772982585974	0	1
46	2	2026-03-08T15:09:46.988875	1772982586988	0	1

Figure: DB table showing one dropped metric on every sample

9.4 Deterministic Host UUID Design

The original schema used a SQLite auto-increment integer as the host primary key. When .db files from two different machines were merged for training data collection, primary key conflicts occurred. The fix was to replace the auto-increment key with a deterministic UUID generated as uuid5(NAMESPACE_DNS, f'{hostname}-{mac_int}'). The same machine now always produces the same host_id, making cross-machine .db merges conflict-free.

9.5 GPU Absence Handling

Systems without a NVIDIA GPU need to run the application without any GPU-related errors. This was addressed at three levels: the GPU collector returns an empty list and sets gpu_available = False when NVML initialisation fails; the Storage Service skips GPU inserts when no GPU data is present; the Analytics Service substitutes zero values for all GPU features when gpu_available is False. The GPU panel in the Live Monitor is left blank rather than showing placeholder values. This was verified on the school computer in the test set that had no GPU.

9.6 pyqtgraph Graph Stutter

During Semester 1 Week 5, the CPU graph was stuttering and felt unresponsive even when the collector itself was fast. The cause was a call to plotItem.setXRange(0, 49, padding=0) on every update tick, which forced pyqtgraph to recalculate and redraw the entire x-axis range on each refresh rather than just updating the curve data. Removing that call and switching to only updating the curve data via setData() eliminated the stutter. A configurable refresh rate was also added through the Settings panel at the same time, allowing the update frequency to be reduced on slower hardware.

9.7 CPU Temperature - Deferred Feature

CPU temperature was listed in the original project architecture diagram as a planned metric. During Semester 1 research, it was established that reading CPU temperature on Windows is hardware-dependent: there is no universally available Windows API for this metric. Different CPU manufacturers and motherboard vendors expose temperature sensors in different ways, with no single library reliably covering all hardware. The WMI approach requires administrator privileges and does not work on all machines; the LibreHardwareMonitor approach requires a kernel driver.

The decision was made to defer CPU temperature collection and mark it as a planned future research path, rather than ship an unreliable implementation that works on some machines and fails silently on others. The final submitted application does not include CPU temperature. All other CPU metrics (utilisation and frequency) are reliably available via psutil and Windows PDH and are included.

During Easter break, while exploring win32pdh further, a thermal zone counter called High Precision Temperature was found via the Windows PDH API and pulled a reading of 61.1°C. It looked promising initially because it was accessible without administrator privileges and returned a plausible temperature value. After researching what the counter actually maps to, it turned out to be a motherboard ACPI sensor rather than the actual CPU die temperature. It reflects the ambient temperature near the board, not the core temperature that would be relevant for throttle detection. Since it does not

reflect real core temps and the throttle label rule does not use temperature anyway, it was ruled out along with the other approaches.

9.8 RAM Graph Hover Bug

During Semester 1 Week 7, after adding the RAM graph, the hover indicator had a rendering bug. Figure 9.8 shows the issue: when the mouse moved over the graph, the red dot was being drawn at the correct nearest data point, but the previous dot was not being removed. Between graph refresh cycles, every mouse movement added a new dot without clearing the last one, so they accumulated on screen. The stacking cleared itself when the graph updated on the next refresh cycle, which wiped the canvas. The fix was to explicitly remove the previous dot before drawing the new one on each hover event, so only one dot is ever visible at a time regardless of how fast the mouse moves between updates.

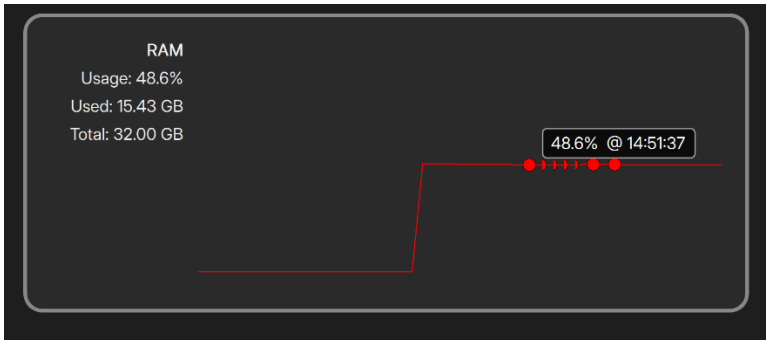


Figure 9.8: Graphical dot issue

9.9 GPU Library Change: GPUtil to pyNVML

The initial GPU data collection implementation used the GPUtil library, which was the most commonly referenced Python GPU monitoring library in tutorials and documentation at the time of research. During Semester 1 Week 10, a Reddit thread highlighted that GPUtil wraps nvidia-smi internally, adding process-spawning overhead to every collection call, and exposes a more limited set of metrics than the underlying NVML interface. Investigation led to nvidia-ml-py (pynvml), which is NVIDIA's own official Python binding to the NVML C library, providing lower overhead and a broader metric set. The switch was made and documented in the project log.

10. Ethics

10.1 Data Privacy

All performance data collected by the application is stored exclusively in a local SQLite database on the user's own machine. No data is transmitted to any external server, cloud service, or third party at any point. The .db file contains hardware identifiers including hostname, MAC address, and OS version, alongside the hardware metrics collected during each session. The user has full control over this data at all times and can delete the telemetry.db file at any point without affecting the application. No account, login, or internet connection is required at any stage.

10.2 AI Use Disclosure

AI tools were used for several categories of minor task throughout the project, all in line with the module AI use policy and documented in Appendix A. This included debugging assistance, where error messages or unexpected behaviour were described to an AI tool to help narrow down likely causes, which were then independently investigated and verified before any fix was applied. AI was also used for technology research, comparing libraries and approaches before making design decisions, with all comparisons cross-checked against official documentation. Unfamiliar code snippets found online were sometimes explained using AI before being adapted, and in Semester 2 Week 2, ChatGPT was used to generate a populated .db file for schema validation testing. GitHub Copilot was used occasionally during implementation for inline suggestions, all of which were manually reviewed and none accepted directly.

All architecture decisions, feature engineering, label design, model training, and core implementation are my own work. I can explain and demonstrate every part of the project independently.

10.3 System Monitoring Considerations

The application has no process-level visibility. It only collects aggregate CPU, RAM, GPU, and Disk figures across the whole system, not per-application or per-user breakdowns. This means it cannot identify which specific program or user is causing a particular reading. Even on a shared machine, the tool provides no more information than what Windows Task Manager already exposes to any logged-in user, and unlike Task Manager it has no network capability or remote access of any kind.

10.4 Accessibility and Plain-Language Design

A deliberate design goal of this project was to make hardware diagnostics accessible to users regardless of their technical background. Existing tools present raw sensor values, temperatures in degrees, frequencies in MHz, throughput in bytes per second, with no interpretation. This is useful to an expert but meaningless to most users.

The Analytics Panel addresses this by presenting detected faults in plain language with a brief description and a suggested fix. Rather than displaying a fault label and a confidence score, the panel tells the user what is happening and what they can do about it. For example, instead of showing gpu_overheating: 91%, it displays "GPU

Overheating: your graphics card is running at a dangerously high temperature. Make sure your PC case has good airflow and clean dust from the GPU fans." This makes the application useful to someone with no hardware knowledge, not just technically experienced users.

It should be noted that all descriptions and fix suggestions are written in English only. The application does not currently support other languages, which limits accessibility for non-English speaking users and is something that could be addressed in a future version.

11. Conclusion

This project set out to build a Windows desktop application that monitors hardware in real time, stores all telemetry locally, and uses machine learning to detect hardware faults and report them in plain language. The goal was to serve both technically experienced users who want detailed hardware data and everyday users who just want to know if something is wrong, a gap that existing tools do not address.

The Smart System Performance Dashboard achieves all seven functional requirements. The four-service pipeline, Collector, Storage, Analytics, and Dashboard, operates as a complete end-to-end system. Hardware metrics are collected every second, persisted to a normalised 10-table SQLite database, fed into a Random Forest classifier every 5 seconds, and displayed in a PyQt6 desktop application. Collection cycles complete in 22 to 38ms, well within the 50ms target. The classifier, trained on over 20,000 samples using a rule-based labelling approach, achieves an average F1 score of 0.98 across all 12 fault categories. Over 80 automated tests across three suites verify every subsystem independently.

From an engineering standpoint, the project demonstrates how a well-designed modular architecture, where each service has clear boundaries and can be developed, profiled, and tested in isolation, makes a complex multi-component system manageable. The 95% reduction in collection cycle time achieved through structured root cause analysis, and the SQLite schema design that allows continuous reads and writes without contention, are examples of engineering decisions that had a direct and measurable impact on the final product. The combination of hardware systems programming, database design, and machine learning in a single cohesive local application is the aspect of the project I am most satisfied with.

12. Future Improvements

Several extensions were identified during development that fell outside the project scope. CPU temperature monitoring was investigated and deliberately deferred, there is no universal Windows API for CPU die temperature, and shipping an unreliable implementation was considered worse than omitting the metric entirely.

A historical trend view would require no backend changes, as all telemetry is already stored and timestamped in SQLite. Cross-platform support is also achievable without structural changes, as only the collector layer uses Windows-specific APIs. Finally, packaging the application with PyInstaller into a standalone executable would make it accessible to non-technical users without a Python installation.

References

[1] Qt Group, 'Getting Started with Qt for Python', Qt Documentation, 2025. [Online]. Available: http://doc.qt.io/qtforpython-6/gettingstarted.html

[2] pyqtgraph, 'pyqtgraph Qt Crash Course', pyqtgraph Documentation, 2025. [Online]. Available: https://pyqtgraph.readthedocs.io/en/latest/getting_started/qtcrashcourse.html

[3] PyShine, 'Stream 45: Live Data Visualization using PyQtGraph and QTimer', YouTube, 2020. [Online]. Available: https://www.youtube.com/watch?v=FDoEpYGA93E

[4] BugBytes, 'PyQt6 - Build Desktop Apps with Python - Intro and Core Concepts!', YouTube, 2024. [Online]. Available: https://www.youtube.com/watch?v=Lx-kfm5jCUw

[5] Normalized Nerd, 'Random Forest Algorithm Clearly Explained!', YouTube, 2021. [Online]. Available: https://www.youtube.com/watch?v=v6VJ2RO66Ag

[6] NVIDIA, 'pynvml Tutorial', Kaggle, 2025. [Online]. Available: https://www.kaggle.com/code/currypurin/pynvml-tutorial

[7] G. Rodola, 'psutil Documentation', Read the Docs, 2025. [Online]. Available: https://psutil.readthedocs.io/en/latest/

[8] NVIDIA, 'NVML API Reference Guide', NVIDIA Developer Documentation, 2025. [Online]. Available: https://docs.nvidia.com/deploy/nvml-api/

[9] scikit-learn Developers, 'sklearn.ensemble.RandomForestClassifier', scikit-learn 1.8.0 Documentation, 2025. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html

[10] scikit-learn Developers, 'sklearn.multioutput.MultiOutputClassifier', scikit-learn 1.8.0 Documentation, 2025. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.multioutput.MultiOutputClassifier.html

[11] SQLite Consortium, 'Write-Ahead Logging', SQLite Documentation, 2025. [Online]. Available: https://www.sqlite.org/wal.html

[12] Microsoft, 'Performance Counters (Windows Performance Data Helper)', Windows Dev Docs, 2025. [Online]. Available: https://learn.microsoft.com/en-us/windows/win32/perfctrs/performance-counters-portal

[13] M. Maini and S. Sabri, 'Why Machine Learning Matters', Medium - Machine Learning for Humans, 2017. [Online]. Available: https://medium.com/machine-learning-for-humans/why-machine-learning-matters-6164faf1df12

[14] Serokell, 'Random Forest Classification Algorithm Explained', Serokell Blog, 2022. [Online]. Available: https://serokell.io/blog/random-forest-classification

[15] A. Fox, 'Smart System Performance Dashboard - Final Year Project', GitHub, 2025. [Online]. Available: https://github.com/AndrewFoxATU/Final-Year-Project

[16] T. Golden, 'win32pdh', pywin32 Documentation. [Online]. Available: https://timgolden.me.uk/pywin32-docs/win32pdh.html

[17] DbVis Software, 'DbVisualizer', DbVis Software, 2025. [Online]. Available: https://www.dbvis.com/download/

[18] pandas Development Team, 'pandas Documentation', NumFOCUS, 2025. [Online]. Available: https://pandas.pydata.org/

[19] MSI, 'MSI Afterburner', MSI, 2025. [Online]. Available: https://www.msi.com/Landing/afterburner/graphics-cards

[20] CPUID, 'HWMonitor', CPUID, 2025. [Online]. Available: https://www.cpuid.com/softwares/hwmonitor.html

[21] REALiX, 'HWiNFO', REALiX, 2025. [Online]. Available: https://www.hwinfo.com/

[22] NZXT, 'NZXT CAM', NZXT, 2025. [Online]. Available: https://nzxt.com/pages/cam

[23] UL LLC, '3DMark', UL LLC, 2025. [Online]. Available: https://www.3dmark.com/

[24] pytest Developers, 'pytest: helps you write better programs', Read the Docs, 2025. [Online]. Available: https://docs.pytest.org/en/stable/

[25] J. D. Hunter, 'Matplotlib: A 2D Graphics Environment', Matplotlib, 2025. [Online]. Available: https://matplotlib.org/

Appendix

Appendix A - AI Use Declaration

Tool	When	Purpose	How Output Was Used
ChatGPT / AI assistant	Throughout Sem 1 & 2	Debugging assistance. Example prompt: 'I'm getting this error: [error]. What are the most likely causes and fixes?'	Used to identify potential root causes, then independently investigated and fixed. No code accepted directly.
ChatGPT / AI assistant	Sem 1 Weeks 9-11	ML model selection research. Prompted to compare strengths and weaknesses of K-Means, Isolation Forest, Random Forest, and others for this use case.	Used to inform the decision. All comparisons were cross-checked against documentation and the final selection was my own.
ChatGPT / AI assistant	Throughout Sem 1 & 2	Technology selection research. Example prompt: 'Why should I use X over Y for this project? Compare pros/cons.'	Used to gather comparison points, which were then cross-checked with official documentation before making design decisions.
ChatGPT / AI assistant	Throughout Sem 1 & 2	Understanding unfamiliar code. Example prompt: 'Why is ___ used in this code snippet? Explain line-by-line.'	Used to understand examples found online before adapting them. Example prompt: 'How can I adapt this example to my app.' Final implementation is the student's own.
ChatGPT / AI assistant	S2 Week 2	Generated a populated .db file to visualise what real data would look like inside the schema at a point when only the schema existed. Prompt: 'here is the schema for my data-collection database, populate this with realistic system data and send it to me as a .db file'	Used only to inspect the schema visually in DBvisualiser. The file was never used in testing or development.
Claude / AI assistant	Sem 2 report writing	Asked questions about rubric requirements to understand what each section needed to cover. Example: 'what does this rubric criterion mean and am I meeting it?'	Used to understand marking criteria. All writing and content decisions are my own.
ChatGPT / AI assistant	Throughout Sem 1 & 2	Spell and grammar checking written work including weekly logs and report sections.	Used to catch errors. Wording and ideas throughout are my own.
GitHub Copilot	Various	Occasional inline code suggestions during implementation sessions.	All suggestions manually reviewed and adapted. None accepted directly. All architecture decisions, feature engineering, and label design are my own work.

Appendix B - Full Label Threshold Specification

The following table documents the exact threshold values used in analytics_service/analytics/labels.py for each of the 12 fault labels.

Label	Component	Threshold Conditions (from labels.py)
cpu_thermal_throttle	CPU	cpu_percent_total > 70 AND freq_current_mhz_roll_slope < -50
cpu_bottleneck	CPU	cpu_percent_total > 90 AND ram_usage_percent < 70 AND disk_usage_percent < 50
cpu_sustained_high_load	CPU	cpu_percent_total_roll_mean > 88
ram_pressure	RAM	ram_usage_percent > 80 AND cpu_percent_total < 50
ram_memory_leak	RAM	ram_usage_percent_roll_slope > 0.3 AND swap_usage_percent_roll_slope > 0.05
excessive_swap_usage	RAM	swap_usage_percent > 50
disk_full	Disk	disk_usage_percent > 90
disk_bottleneck	Disk	cpu_percent_total < 40 AND ram_usage_percent < 60 AND (read_speed_bytes > 200,000,000 OR write_speed_bytes > 200,000,000)
disk_high_latency	Disk	avg_read_latency_ms > 20 OR avg_write_latency_ms > 20
gpu_overheating	GPU	gpu_temp_c > 85
gpu_power_throttle	GPU	gpu_power_usage_w >= 0.98 * gpu_power_limit_w AND gpu_core_clock_mhz_roll_slope < -5
gpu_vram_pressure	GPU	gpu_mem_util_percent > 90

Appendix C - Software Dependencies

The application was developed and tested with the following library versions (requirements.txt):

Library	Version	Purpose
PyQt6	6.7.0	GUI framework - main window, widgets, threading
pyqtgraph	0.13.4	Real-time scrolling graphs with GPU acceleration
psutil	6.0.0	CPU, RAM, and Disk data collection
pywin32	307	Windows PDH counters for accurate CPU frequency
nvidia-ml-py	12.560.30	pynvml - NVIDIA GPU metrics via NVML

Library	Version	Purpose
scikit-learn	1.8.0	RandomForestClassifier, MultiOutputClassifier
pandas	3.0.2	Training data CSV handling
numpy	2.1.2	Feature array operations, polyfit for rolling slope
matplotlib	3.10.1	Training analysis visualisations (not used in the app)
pytest	9.0.2	Automated test runner
loguru	0.7.2	Structured application logging